

UNIVERSITY OF CAMBRIDGE
Department of Engineering

Adjoint-based optimization of hydrofoils

Author name: Yana Lishkova
Supervisor: Professor Matthew Juniper
Date: 29 May 2019

I hereby declare that, except where specifically indicated, the work submitted herein is my own original work.

Signed:_____ Date:_____

Adjoint-based optimization of hydrofoils

Technical Abstract

Yana Valentinova Lishkova
29 May 2019

In this project we explored the use of automatic differentiation in the context of 2D airfoil shape optimisation. For this purpose three different flow solvers were developed for the calculation of the airfoil's lift-to-drag ratio and its gradient with respect to the input geometry parameters. These solvers were then incorporated within a limited-memory implementation of the Broyden-Fletcher-Goldfarb-Shanno algorithm for bound constrained optimisation (L-BFGS-B) in order to estimate the maximum obtainable lift-to-drag ratio given a Reynolds number and an angle of attack specified by the user.

The solvers, named FDM, FAD and RAD, all have similar architecture developed entirely in Fortran 90. To define the airfoil geometry the user can select one of three methods: direct surface coordinates specification, an implementation of the van de Vooren mapping technique or the "class function/shape function" transformation technique (CST). To model the resulting flowfield and obtain the airfoil's lift-to-drag ratio all three solvers incorporate a linear strength vortex panel method coupled with an integral boundary layer model. The solvers differ in the method of obtaining the gradient of the lift-to-drag ratio with respect to the input geometry parameters. The FDM solver approximates the partial derivatives composing the gradient using the finite difference method (FDM), whereas the FAD and RAD solvers implement automatic differentiation in forward or reverse mode respectively using line by line source code differentiation.

To investigate the optimisation problem the solvers were separately implemented within the L-BFGS-B algorithm and their performance was compared for three different design spaces. Under the van de Vooren two-dimensional parametrisation the implementation of both the FDM and the RAD solver resulted in similar convergence behaviour, however the overhead of memory allocation required by the RAD solver resulted in longer run time for a single gradient evaluation compared to the FDM solver. Defining the airfoil shape using six and ten parameters under the CST parametrisation revealed the benefits of using reverse mode automatic differentiation. The RAD solver was able to maintain a constant run time per single gradient calculation regardless of the dimension of the input vector whereas the FDM solver showed increasing run time. Additional sensitivity studies showed that the FAD and RAD solver compute a derivative equal to the exact derivative, whereas the truncation error in the FDM derivative approximation is strongly dependent the perturbation step size. For this reason, truncation errors in the partial derivatives building the gradient resulted in inaccurate search directions and often led to the termination of L-BFGS-B at local maxima much lower than the values obtained by the RAD and FAD solvers.

Contents

1 Background **3**
- 1.1 Iterative design . . . 3
- 1.2 Inverse design . . . 3
- 1.3 Numerical optimisation . . . 4
- 1.4 Gradient estimation . . . 6
  - 1.4.1 Finite difference method . . . 7
  - 1.4.2 Complex-step derivative approximation . . . 7
  - 1.4.3 Automatic differentiation . . . 8
  - 1.4.4 Comparison of methods . . . 10
- 1.5 Related work . . . 11
- 1.6 Importance of lift-to-drag ratio . . . 12
- 1.7 Aims of work . . . 13

2 Implementation **14**
- 2.1 Tool selection . . . 14
- 2.2 Flow solver . . . 14
  - 2.2.1 Geometry definition . . . 15
  - 2.2.2 2D potential flow solver . . . 18
  - 2.2.3 Integral boundary layer solver . . . 20
- 2.3 Gradient estimation . . . 22
  - 2.3.1 TAPENADE . . . 22
  - 2.3.2 Challenging aspects . . . 22
  - 2.3.3 Developed Solvers . . . 24
- 2.4 Optimisation . . . 26
  - 2.4.1 Problem statement . . . 26
  - 2.4.2 Unconstrained optimisation . . . 26
  - 2.4.3 Constrained optimisation . . . 27
- 2.5 Full framework overview . . . 28

3 Results and discussion **30**
- 3.1 Flow solver validation . . . 30
  - 3.1.1 Sensitivity analysis through forward mode AD . . . 31
- 3.2 Optimisation . . . 34
  - 3.2.1 Optimisation in two dimensions . . . 34
  - 3.2.2 Optimisation in six dimensions . . . 38
  - 3.2.3 Optimisation in ten dimensions . . . 39

4 Future work **42**
- 4.1 Model improvement . . . 42
- 4.2 Gradient calculation . . . 42
- 4.3 Optimisation algorithm . . . 43

5 Conclusion **43**

Appendices **46**

A Influence coefficients **47**

B Laminar regime **48**

C Turbulent regime **49**

D Risk assessment retrospective **50**

List of Figures

2.1 Example van de Vooren geometries for $l = 1$ and a as specified in Eqn. 2.4 16
2.2 Example CST geometries for $N_1 = 0.5$ and $N_2 = 1$ 18
2.3 A single panel notation 18
2.4 Shape of the scaled *tanh* function used to construct the logical statements 23
2.5 Internal structure and data flow within the RAD solver 25
2.6 Flow chart of the full optimisation process 29

3.1 Comparison between experimentally measured data and computed estimates for NACA 0012 at $Re = 3 \times 10^6$ 30
3.2 Comparison of the error in first derivative computation resulting from the FDM and FAD solvers for a range of step sizes for all 402 control parameters. The largest values with $\Delta = 10^{-4}$ have been cut off from the figure to allow for the straight line relation to be shown 33
3.3 Representation of the design spaces 34
3.4 Optimisation path for $Re = 3 \times 10^6, \alpha = 10^\circ$ starting from $k = 1.2, \epsilon = 0.7$ 35
3.5 Optimisation results for $Re = 3 \times 10^6, \alpha = 10^\circ$ 36
3.6 Comparison between the convergence behaviour of the FDM and RAD solver implemented in the L-BFGS-B algorithm starting from initial parameter values $k = 1.2$, $\epsilon = 0.4$ 36
3.7 Optimisation results for $Re = 3{\times}10^6, \alpha = 10^\circ$ and $\mathbf{w} = [-1,\ -1,\ -1,\ 1,\ 1,\ 1]^T$ 38
3.8 Shape iteration using the RAD solver with a new initial parameter vector . 39
3.9 Optimisation results for $Re = 3 \times 10^6, \alpha = 10^\circ$ 40

A.1 A single panel notation 47

Chapter 1

Background

1.1 Iterative design

Prior to the 1960s, wind tunnel testing was the main tool for aerodynamic analysis and design. A combination of measurements and visualisation techniques allowed engineers to investigate and collect data on a set of standard geometries which were then applied in industry. The advent of digital computing presented many new opportunities for scientific research. In many fields, including fluid dynamics, the numerical solution of the established physical principles was impossible with the current state of technology and thus required new mathematical models and algorithms to be developed to enable robust and efficient computer simulation and investigation. Pioneers of this approach in the field of fluid dynamics include the work at Los Alamos National Lab under the supervision of F.H. Harlow [1] and the introduction of panel methods starting with the work of J. Hess and A.M. Smith [2].

Today high-performance computing has allowed computational analysis to become an intrinsic part of the design of airfoil and hydrofoil shapes. The design process remains iterative: designs are created, validated through experiment, analysed and then adjusted until acceptable values of the observed characteristics are reached. In industry, it is unlikely that this procedure will yield an optimised design before the project runs out of time or finances.

Furthermore, computational simulation and analysis requires substantial computing resources, which can be prohibitive for its use in the early stages of designs development or for optimisation studies where insight in the dependence of the flow characteristics on the design parameters are more important than highly-accurate and detailed analysis.

1.2 Inverse design

For this reason researchers have been working on different approaches in order to explore the design space more efficiently and shorten design cycle times. Early attempts included inverse design methods which aimed at direct computation of the geometry given a desired surface pressure or velocity distribution. In the framework of airfoil shape optimisation a pioneering inverse approach was developed by Lighthill in 1945 [3], who analysed 2D potential flow by conformally mapping the profile to a unit circle.

Inverse methods however are applicable to only a limited number of problems for which the designer has prior knowledge of the flow field and the pressure distribution required to

optimise the relevant performance criterion. In many cases attempts to enforce physically unrealisable pressure distribution can lead to ill-posed problems.

1.3 Numerical optimisation

To solve the inverse design problem and do so computationally, one can treat the problem as a special case of the numerical optimisation problem. In this mathematical framework the problem can be posed as follows:

$$\min_{\mathbf{x}} f(\mathbf{x}) \tag{1.1}$$

$$\text{subject to: } \mathbf{g}(\mathbf{x}) = 0$$
$$\mathbf{c}(\mathbf{x}) \leq 0$$

where $\mathbf{x}$ is a vector of design parameters, f is the objective function - the performance criteria, which we aim to minimise or maximise, and $\mathbf{g}$ and $\mathbf{c}$ are vector functions, describing the constraints placed on $\mathbf{x}$. In the context of airfoil shape optimisation, f is the airfoil's performance criteria that we would like to optimise and $\mathbf{x}$ defines the geometry of the airfoil as will be discussed in Section 1.6.

Many different algorithms for solving this general problem exist and their applicability depends on the presence and definition of the constraints as well as the behaviour of the objective function within the design space they delineate. Most optimisation algorithms are iterative in nature. Given an initial guess of the solution to the optimisation problem $\mathbf{x}_0$, the algorithm iteratively looks for an $\mathbf{x}$ value which maximises/minimizes $f(\mathbf{x}_0)$.

The search for the new iteration of $\mathbf{x}$ follows one of two main strategies: trust region or line search method. In the trust region method a model m_i of the objective function is created, which approximates its behaviour in the proximity of the current iteration point $\mathbf{x}_i$. Because for general nonlinear functions linear and quadratic approximations model the objective function only locally, a trust region is defined within a distance of radius of r_i. A search direction and step α along it that result in the best possible improvement of m within this region are then selected such that

$$\alpha = \operatorname{argmin} m_i(\mathbf{x}_i + \alpha) \quad \text{such that} \quad (\mathbf{x}_i + \alpha) \in \text{the trust region} \tag{1.2}$$

If this step proves to be unsatisfactory, because the improvement it introduces is too small or even negative, then the model is believed to be a bad approximation of the function and the radius r_i must be decreased to obtain a better one. Conversely, if the current model results in an acceptable improvement, the trust region can be enlarged and the procedure is repeated again [4, 5].

In line search we do the reverse and first determine a direction $\mathbf{d}_i$. We then identify an

appropriate step length λ_i such that

$$\lambda_i = \text{argmin } f(\mathbf{x}_i + \lambda_i \mathbf{d}_i), \quad \text{for} \quad \lambda_i > 0 \tag{1.3}$$

This is again an iterative optimisation problem executed at each new iteration i of the overarching optimisation of $f(\mathbf{x})$ and computation of λ_i can thus be expensive. Alternatively, line search algorithms generate a finite number of trial step lengths until one is found which approximates the minimum of Equation 1.3. The new point $\mathbf{x}_{i+1}$ is then computed and a new search direction and step are chosen appropriately [5]. The search direction can be obtained using several different methods with varying levels of accuracy and complexity.

Steepest descent method: This method is a first-order line search method in which the search direction is defined to be in the direction of the negative gradient of the objective function evaluated at the current value of $\mathbf{x}$. Thus for minimisation problems $\mathbf{d}_i = -\nabla f_i$ or alternatively $\mathbf{d}_i = \nabla f_i$ for finding a maxima, where f_i denotes $f(\mathbf{x}_i)$. Combining this with a line search algorithm with a step size λ_i, at each iteration the new solution is given by:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \lambda_i \nabla f_i \tag{1.4}$$

The method avoids the computation of second derivatives but demonstrates slow convergence for many problems (see [5]).

Newton's method: A different method which can be applied to the optimisation of twice-differentiable objective functions is Newton's method. The search direction is obtained from the minimisation of a quadratic model of the objective function and as shown by [5] can be expressed as follows:

$$\mathbf{d}_i = -(\nabla^2 f_i)^{-1} \nabla f_i \tag{1.5}$$

Thus

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \lambda_i (\nabla^2 f_i)^{-1} \nabla f_i \tag{1.6}$$

The direction is reliable when the quadratic model approximates the objective function well and can be used for line search when the Hessian is positive definite, which guarantees the existence of its inverse. Modifications to the Newton method exist to handle problems where this condition is not satisfied (see [5]). Newton based optimisation methods typically have much faster rate of convergence than gradient descent methods, in many cases quadratic. The main drawback, however, is the need to compute the Hessian, which depending on the problem can be an expensive process. Another drawback is that the method usually diverges if the starting point $\mathbf{x}_0$ is too far from the solution [5].

Quasi-Newton methods: In this family of methods, the Hessian matrix ($\nabla^2 f_i$) is not computed and is instead approximated using updates based on gradient evaluations. The Hessian approximation B_i uses the fact that the Hessian can be viewed as the first derivative of the gradient and can thus be approximated using the first order Taylor approximation for the gradient [5]:

$$\nabla f_{i+1} = \nabla f_i + \nabla^2 f_{i+1}(\mathbf{x}_{i+1} - \mathbf{x}_i) + o(\|\mathbf{x}_{i+1} - \mathbf{x}_i\|) \tag{1.7}$$

or equivalently

$$\nabla^2 f_{i+1}(\mathbf{x}_{i+1} - \mathbf{x}_i) \approx \nabla f_{i+1} - \nabla f_i \tag{1.8}$$

Therefore we can define B_i such that:

$$B_{i+1}\mathbf{s}_i = \mathbf{y}_i \tag{1.9}$$

where

$$\mathbf{s}_i = \mathbf{x}_{i+1} - \mathbf{x}_i \qquad\qquad \mathbf{y}_i = \nabla f_{i+1} - \nabla f_i \tag{1.10}$$

Apart from Equation 1.9, which is known as the secant equation, we impose additional requirements on the Hessian approximation: B_{i+1} must be symmetric, positive definite and its difference with respect to B_i must be small and low rank [5]. The new direction based on this approximation can be obtained by replacing the exact Hessian in equation 1.5 as follows:

$$\mathbf{d}_i = -B_i^{-1}\nabla f_i \tag{1.11}$$

Thus

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{s}_i \tag{1.12}$$

Different methods exist for updating B_i. On of the most popular approaches is based on the Broyden–Fletcher–Goldfarb–Shanno algorithm (BFGS) and expressed as follows:

$$B_{i+1} = B_i - \frac{B_i\mathbf{s}_i\mathbf{s}_i^T B_i}{\mathbf{s}_i^T B_i \mathbf{s}_i} + \frac{\mathbf{y}_i\mathbf{y}_i^T}{\mathbf{y}_i^T\mathbf{y}_i} \tag{1.13}$$

Practical modifications of this methodology, alternative update formulations,as well as additional methods for defining the search direction can be found in [5].

1.4 Gradient estimation

In all methods described above, the gradient of the objective function ∇f_i at the current value of x_i must be computed at each iteration i of the optimisation process. Thus an efficient and accurate method for computing the partial derivatives building the gradient is crucial for the optimisation process to succeed.

1.4.1 Finite difference method

A common approach to approximate these derivatives uses the finite difference method (FDM), which is derived by truncating a Taylor series expansion of the function about a point z. The derivative can thus be approximated using the forward or backward difference formulas:

$$\frac{\partial f_i}{\partial z_i} = \frac{f_{i+1} - f_i}{\Delta} + O(\Delta) \tag{1.14}$$

$$\frac{\partial f_i}{\partial z_i} = \frac{f_i - f_{i-1}}{\Delta} + O(\Delta) \tag{1.15}$$

where Δ is the finite-difference interval, also know as the perturbation step size. Placing this formulation within the numerical optimisation framework presented in 1.1, z represents one of the input variables within the vector x and i represents the iteration number during the execution of the optimisation algorithm. From Equations 1.14 and 1.15 we can see that that the forward and backward difference suffer from a truncation error proportional to the perturbation step size Δ and are thus only first-order accurate.

Alternatively, a better second-order approximation of the derivative is obtained by the central difference formula:

$$\frac{\partial f_i}{\partial z_i} = \frac{f_{i+1} - f_{i-1}}{2\Delta} + O(\Delta^2) \tag{1.16}$$

Naturally one would like to reduce the step size to decrease the truncation error. Making it too small, however, introduces a subtractive cancellation error due to representation of numbers in computers [6, 7]. Finding the most appropriate step size requires additional executions of the program and even then the computed derivatives are only approximations. Furthermore, for n input parameters ($\mathbf{x} \in R^n$) at each iteration of the overarching optimisation algorithm the gradient calculation requires $n + 1$ functional evaluations for the forward or backward difference formulas or $2 \times n$ for the central difference approximation. For problems with large number of input parameters the gradient calculation can thus result in great computational cost [5].

1.4.2 Complex-step derivative approximation

The complex-step derivative approximation is an alternative approach which overcomes the subtractive cancellation errors. It was first introduced by Lyness and Moler [8] and can again be derived through the Taylor expansion of a real function f about a real point z. Rather than using a real step Δ, a purely imaginary step $j\Delta$ is adapted as follows:

$$f(z_i + j\Delta) = f(z_i) + j\Delta\frac{\partial f(z_i)}{\partial z_i} - \Delta^2\frac{1}{2!}\frac{\partial^2 f(z_i)}{\partial z_i^2} - j\Delta^3\frac{1}{3!}\frac{\partial^3 f(z_i)}{\partial z_i^3} \tag{1.17}$$

where $j = \sqrt{-1}$. Taking the imaginary parts of both sides of this equation and dividing by Δ results in the following second-order approximation as shown by [9]:

$$\frac{\partial f_i}{\partial z_i} = \frac{\mathrm{Im}[f(z_i + j\Delta)]}{\Delta} + O(\Delta^2) \tag{1.18}$$

The gradient evaluation requires n functional evaluations at each iteration with a truncation error of $\mathrm{O}(\Delta^2)$ in the derivative estimation. Additionally, this methodology requires the use of solvers capable of dealing with complex numbers [9].

1.4.3 Automatic differentiation

Automatic differentiation (AD) is an analytic and computational method for estimation of the derivatives based on the chain rule of differentiation, which does not suffer from the accuracy loss due to truncation error or subtractive cancellation. In automatic differentiation every line of the original program S is considered as primitive operation computing an intermediate value [10]. For example:

$$a = b + c = f_1(b, c) \tag{1.19}$$

$$d = a^2 = f_2(a) \tag{1.20}$$

Thus

$$d = f_2(f_1(b, c))) \tag{1.21}$$

In general any function f can be decomposed as

$$f = f_N \circ f_{N-1} \circ ... \circ f_1 \tag{1.22}$$

where N is the number of primitive operations represented by $\circ$ and the data flow is from right to left. Automatic differentiation uses this primitive operation formulation by exploiting the chain rule of differentiation to obtain the Jacobian of a f with respect to an input vector $\mathbf{x}$ ($J = f'(\mathbf{x})$ or f' for short):

$$f' = f'_N(R_{N-1}) \cdot f'_{N-1}(R_{N-2}) ... \cdot f'_2(R_1) \cdot f'_1(R_0) \tag{1.23}$$

for a vector of input parameters $\mathbf{x}$ and $R_N = f_N(R_{N-1})$, where $R_0 = \mathbf{x}$ [11]. Each of these derivatives (f'_N) on its own is a Jacobian matrix. Computing the full Jacobian however could be an expensive operation requiring large matrices to be stored. Alternatively one can compute only projections of this matrix [11].

1.4.3.1 Forward mode AD

In the forward mode (FAD), also known as the tangent mode, we can compute the "sensitivities" [11]. Given an output Z such that $Z = f(\mathbf{x})$, we can obtain first-order variation in the output $\dot{Z}$ (the sensitivity) given a perturbation $\dot{\mathbf{x}}$ in the input space. Computing the

sensitivity is thus equivalent to computing the directional derivative, which can be defined in terms of the Jacobian matrix as

$$\dot{Z} = J \cdot \dot{\mathbf{x}} \tag{1.24}$$

or written in the above formulation

$$\dot{Z} = f'_N(R_{N-1}) \cdot f'_{R-1}(R_{N-2}) ... \cdot f'_2(R_1) \cdot f'_1(R_0) \cdot \dot{\mathbf{x}} \tag{1.25}$$

as shown by [11]. To be computationally efficient one must compute it from right to left resulting in *matrix* $\cdot$ *vector* operations rather than *matrix* $\cdot$*matrix*. In terms of the example above for every intermediate value an additional variable of the same size, called its differential, will be computed:

$$\dot{a} = \dot{b} + \dot{c} = f'_1 \tag{1.26}$$

$$a = b + c = f_1(b, c) \tag{1.27}$$

$$\dot{d} = 2\dot{a} = f'_2(a) \tag{1.28}$$

$$d = a^2 = f_2(a) \tag{1.29}$$

$$d = f_2(f_1(b, c))) \tag{1.30}$$

And equivalent to Equation 1.25

$$\dot{d} = f'_2(f_1(\mathbf{x})) \cdot f'_1(\mathbf{x}) \cdot \dot{\mathbf{x}} \tag{1.31}$$

where $\dot{\mathbf{x}} = [\dot{b},\ \dot{c}]^T$. Following this procedure of defining elementary functions for every line of the source code and computing their differential results in a differentiated source code S'. Notably, following Equation 1.25 from right to left, the differentials are used in the order they are computed.

1.4.3.2 Reverse mode AD

In reverse mode AD (RAD), also known as the adjoint mode, we compute the gradient with respect to $\mathbf{x}$, which is a symmetric concept to the directional derivative and can be defined as follows:

$$\bar{\mathbf{x}} = J^T \cdot \bar{Z} \tag{1.32}$$

and shown by [11]. The bar notation is standard for reverse AD and represents the perturbation of the variable. Put simply, in forward mode we perturb each input variable in turn to observe the change in the output, whereas with reverse mode we perturb only the output to observe the changes of all the initial variables required for this change in the

output. We do so by reversing the chain rule backwards:

$$\bar{x} = f_1'^T(R_0) \cdot f_2'^T(R_1) ... \cdot f_{N-1}'^T(R_{N-2}) \cdot f_N'^T(R_{N-1}) \cdot \bar{Z} \tag{1.33}$$

Similarly to forward AD, for this operation to be computationally efficient one must compute it from right to left. The issue is that the intermediate values are used in a reverse order to which they are being computed. Thus programs implementing reverse AD must first do a functional evaluation (forward run of S, obtaining Z), storing all intermediate values and only then compute the gradient. Although this can result in higher memory storage requirements, one can show that the computation cost of evaluating the gradient with reverse mode AD is a small multiple of the computation cost for evaluating the original function f and is therefore independent of the dimension of $\mathbf{x}$ [11].

1.4.3.3 Implementation

There are two main approaches to implementing automatic differentiation. The method of differentiating each line of the original program S resulting in a transformed and differentiated source code is known as transformation based AD. Alternatively, one can use overloading, a programming technique which consists of redefining the semantics of basic operations (such as arithmetic operations) according to their arguments. An example of this approach uses dual numbers, where the derivative of each variable is augmented to the number such as in complex numbers. If the language does not support overloading, complex numbers could be used instead [11, 12]. Despite being intuitive and applicable to the computation of not only first, but also second and third derivatives, there are several drawbacks to this implementation of AD. To start, not all languages support overloading or complex numbers. Furthermore, overloading is often poorly optimised by compilers for languages which do and most importantly, it is unsuitable for reverse mode AD [11].

1.4.4 Comparison of methods

Notably the gradient can be evaluated using forward mode AD as well. In applications with a scalar output Z, to do so would require the execution of the full differentiated program S' once for each input parameter, building the gradient one derivative at a time [11]. This is similar to the method of constructing the gradient using the finite difference method, where the number of functional evaluations increases linearly with the dimension of the input vector. Thus forward AD eliminates subtractive cancellation and truncation errors, but the complexity remains a function of the number of inputs. On the other hand reverse mode AD requires a single run of the original program S followed by a run of its differentiated equivalent and as discussed above has computational cost independent of the number of inputs, which also comes with increased accuracy in the derivative computation.

1.5 Related work

Due to the memory storage requirements of reverse mode AD, its implementation is not practical for all applications. Depending on the memory storage approach used, one can show that the memory cost of of reverse mode AD can be proportional to the execution time of the program [11]. Another issue presented by [11] addresses the difficulties of determining the convergence behaviour of the differentiated code for iterative programs. For this reason several attempts to implement automatic differentiation in the field of computational fluid dynamics (CFD) have focused on the combination of adjoint methods with reverse mode AD [13, 14, 11].

The adjoint methods extend on the optimisation formulation presented in Section1.1 using a control theory approach applied to systems described by partial differential equations developed by Lions. In this framework, the governing equations of the flowfield are introduced as constraints, the control is represented by the boundary of the airfoil section and shape changes are driven by optimisation of a certain aerodynamic performance characteristic of the airfoil, such as lift, drag or their ratio. This allows for the inverse design problem to be rewritten in the framework of constrained optimisation as follows:

$$\max_{\mathbf{x}} f(\mathbf{x}, W(\mathbf{x})) \tag{1.34}$$

$$\text{subject to:} \quad g(\mathbf{x}, W(\mathbf{x})) = 0$$

where f is the performance criterion we aim to optimise and W is the flow state, which is dependent on the control parameter vector $\mathbf{x}$, defining the shape of the airfoil. The constraints g are given by the governing partial differential equations describing the flowfield. The adjoint method aims to compute the gradient of f with respect to $\mathbf{x}$. For this purpose we must solve the following total derivative expression:

$$\frac{df}{dx} = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial W}\frac{dW}{dx} \tag{1.35}$$

To evaluate the term $\dfrac{dW}{dx}$ one can use the total derivative expression for the constraints:

$$\frac{dg}{dx} = \frac{\partial g}{\partial x} + \frac{\partial g}{\partial W}\frac{dW}{dx} = 0 \tag{1.36}$$

which can be rearranged as follows:

$$\frac{dW}{dx} = -\Big(\frac{\partial g}{\partial W}\Big)^{-1}\frac{\partial g}{\partial x} \tag{1.37}$$

Combining Equations 1.35 and 1.37 we have

$$\frac{df}{dx} = \frac{\partial f}{\partial x} - \frac{\partial f}{\partial W}\Big(\frac{\partial g}{\partial W}\Big)^{-1}\frac{\partial g}{\partial x} \tag{1.38}$$

which can be rewritten as

$$\frac{df}{dx} = \frac{\partial f}{\partial x} - \lambda^T\frac{\partial g}{\partial x} \tag{1.39}$$

λ is a Lagrangian multiplier,which satisfies the following equation, known as the adjoint equation:

$$\Big(\frac{\partial g}{\partial W}\Big)^T\lambda = \Big(\frac{\partial f}{\partial W}\Big)^T \tag{1.40}$$

Thus in this method at each iteration of the optimisation algorithm, the gradient is computed using one solution of the nonlinear flow governing equations and one solution of the linear adjoint equations [15]. Deriving the adjoint equations is a non-trivial task and must be done individually for each optimisation problem.

An alternative hybrid approach between the adjoint method and reverse mode AD has been demonstrated by [13], [14] and [16]. This methodology implements RAD selectively within a hand-written implementation of a specific flow solver for the adjoint. Thus instead of discretising the adjoint equations, the derivatives within them are identified and computed using RAD, while the iterative resolution of the system is accomplished by the hand-written solver. This leads smaller memory storage requirements in comparison to using reverse mode AD directly to calculate the gradient of the objective function [16].

1.6 Importance of lift-to-drag ratio

Having posed the problem in a mathematical framework for numerical airfoil shape optimisation the natural question arises of which design characteristic should be optimised using iteration of the airfoil shape and how the design space should be defined. The method of geometry definition must delineate a design space which allows efficiency and numerical stability of the optimisation algorithm and permit the existence of meaningful optima [15].

In terms of airfoil performance two dimensionless performance criteria are often used, the lift (C_L) and drag coefficients (C_D), which are defined as the ratio of the lift and drag force respectively to the force produced by the dynamic pressure multiplied by the wing area A:

$$C_L = \frac{F_L}{\frac{1}{2}\rho U^2 A} \qquad C_D = \frac{F_D}{\frac{1}{2}\rho U^2 A} \tag{1.41}$$

where ρ is the air density and U is the free stream velocity [17]. F_L represents the wing's lift, which is the result of the pressure difference between the upper and lower surface of

the airfoil. This is due to the shape of the airfoil resulting in faster flow on the upper than the lower surface.

The drag force F_D is also strongly dependent on the wing's geometry. For a 2D airfoil section it is comprised of two main components. The first is skin friction drag due to viscous effects in the boundary layer and the second is form drag resulting from the normal surface pressure distribution. The shape of the airfoil determines this distribution and thus the behaviour of the boundary layer. Adverse pressure gradients can cause the flow to detach from the surface of the wing resulting in significant drop in lift and increase in drag. Thus careful control of the airfoil surface and the resulting pressure distribution are essential for the performance of the wing [17, 18]. Combining the effect of the airfoil's geometry on these coefficients, designers typically aim to optimise the shape such as to maximise L/D, the lift-to-drag ratio, which is equivalent to maximising C_L/C_D.

1.7 Aims of work

In this project we aim to conduct an initial investigation of the applicability of automatic differentiation for the estimation of the gradient of the lift-to-drag ratio in the context of airfoil-shape optimisation. We aim to compare the performance of forward and reverse mode AD respectively to the finite difference method in terms of sensitivity analysis and gradient estimation for different flow parameters with respect to changes in the airfoil geometry definition.

Previous work within the research group focuses on the application of adjoint method to the estimation of the gradient. We hope this work to be a preliminary study toward the application of reverse AD in a hybrid implementation of the adjoint methods and the basis of further research work within the group.

Chapter 2

Implementation

This chapter presents the implementation of a framework for shape optimisation of 2D wing sections. The aim of the developed software is to allow the user to input an angle of attack (α), Reynolds number (Re) and parameters, defining the geometrical shape of a 2D airfoil and obtain in result a value for the maximum achievable lift-to-drag ratio and the geometric profile which achieves it under the specified conditions of the flow (α and Re). The system architecture consist of three main parts, which will be described in turn and issues during the implementation will be highlighted. The first part describes the development of a flow solver incorporating a 2D potential flow panel method and an integral boundary layer solver. The second part concerns the differentiation of the source program resulting in two solvers for computation of the gradient through forward and reverse automatic differentiation respectively. The final part describes the selection and implementation of an optimisation algorithm.

2.1 Tool selection

The entire framework was developed in Fortran 90 due to the decision to make partial use of the source-to-source transformation tool TAPENADE for generating the differentiated code [19, 20]. The flow solver and the optimisation algorithm were tested through equivalent implementations in MATLAB. This allowed for the results from the developed flow solvers as well as the optimisation framework to be investigated in both Fortran 90 and MATLAB. Extensive use was made of the LAPACK Fortran 90 library [21] for the manipulation of matrices such as the creation of matrix inversion routines and the solution of systems of simultaneous equations. To enable the Store-All memory approach needed by reverse mode AD two external routines provided by TAPENADE were also used [19]. Furthermore, several hand-written modules were developed to enable clear routine differentiation. These included the implementation of Euler and Runge-Kutta explicit numerical methods for solving first-order differential equations used in the boundary layer integral solver. These simplistic implementations were preferred to the use of developed libraries as they allow transparency of the code and thus enable line by line source-code differentiation. All time tests presented in Section 3.2 were carried out on a single 2.5 GHz CPU virtual machine with 2048MB RAM running an Ubuntu operating system.

2.2 Flow solver

The flow solver requires from the user the specification of the flow conditions: the angle of attack, α, and the Reynolds number, Re. Three options for geometry definition are also

present- direct shape coordinates specification or two different parametrisation methods. The solver separates the flow field in an inviscid region and viscous boundary layer. An inviscid 2D potential flow panel method has been chosen to find the pressure coefficients over the upper and the lower surface of the airfoil. In the process the overall circulation is obtained and used to estimate the lift coefficient. The location of the leading edge stagnation point and the pressure distribution are fed to an integral boundary layer solver, which calculates the momentum and energy thickness of the boundary layer, resulting in identification of locations of laminar to turbulent flow transition, separation and possible reattachment and calculation of the drag coefficient. The solver leads to an estimation of the ratio of the lift-to-drag, which will be the objective function in the optimisation framework, developed in this project.

The numerical methods and the overall structure of the code follow a previous MATLAB implementation presented in [17]. This chapter will summarise this framework and highlight the modifications that were carried out to create the solver from scratch in Fortran 90 and enable source code differentiation. The changes include modifications of the boundary solver control flow structure and the integral solver numerical integration methods, both needed to enable the the chain rule differentiation required by reverse mode AD. Additional parametrisation method is also included to allow for higher dimension parametrisation of the airfoil's shape.

2.2.1 Geometry definition

The flow solver allows for two different methods of specifying the geometry, specifying the shape directly through surface point coordinates or using a parametrisation approach. In all cases, once the user has specified the geometry, a high-resolution surface description is generated via cubic spline interpolation. The choice of geometry definition influences the design space and thus the run time of the implemented optimisation algorithm as will be discussed in Section 3.2. Aside from the direct coordinate definition, two parametrisations are available. A third parametrisation was also attempted to obtain airfoils through the Joukowsky transformation. The cusped trailing edge of these airfoils, however , results in an overlap of the upper and lower surface toward the trailing edge leading to numerical difficulties for the implemented 2D potential flow panel method, described in Section 2.2.2 [22]. This problem can be avoided by the introduction of an additional constant in the Joukowski formulation which guarantees finite curvature at the trailing edge, as successfully attempted by [15]. This however results in a limited design space.

2.2.1.1 van de Vooren airfoil

The alternative approach, used in this project, was first introduced by van de Vooren and de Jong [23] and results in the definition of a symmetric airfoil with a finite trailing edge.

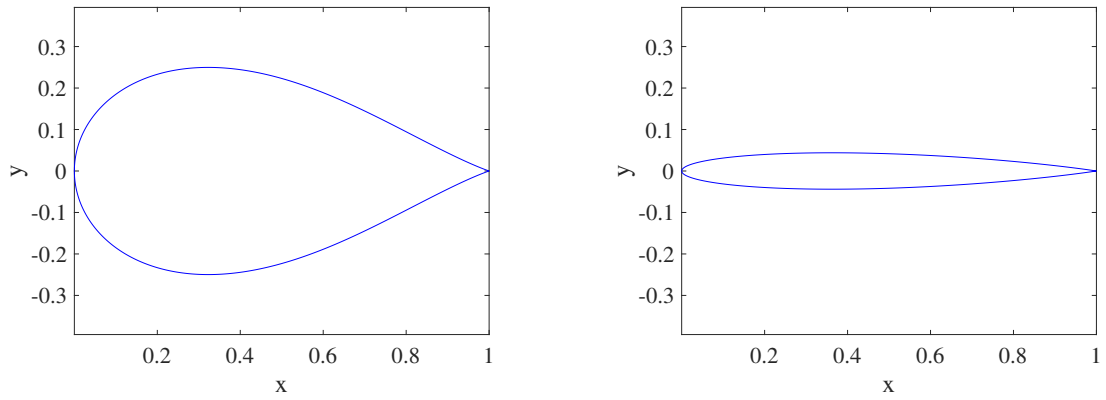
Both this mapping and the Joukowsky transformation allow for the airfoil section belonging to the physical plane $z = x + iy$ to be mapped to a circular cylinder in $\zeta = \xi + i\eta$. This technique, known as conformal mapping, enables exact solutions to the potential flow to be found. For the van de Vooren airfoil the transformation takes the form:

$$z = \frac{(\zeta - a)^k}{(\zeta - a\epsilon)^{k-1}} + l \tag{2.1}$$

and for a circle of radius a in the ζ plane ($\zeta = ae^{i\theta}$) results in a symmetric airfoil with a finite trailing edge angle in the z complex plane as follows:

$$z = a\frac{((cos\theta - 1) + isin\theta)^k}{((cos\theta - \epsilon) + isin\theta)^{k-1}} + l \tag{2.2}$$

To define the airfoil shape, the user specifies three main parameters: the thickness of the airfoil ϵ, the chord length l and the the trailing edge angle β through the k parameter as $\beta = \pi(2 - k)$.



(a) Airfoil section with $k = 1.8$ and $\epsilon = 0.3$ (b) Airfoil section with $k = 1.9$ and $\epsilon = 0.03$

Figure 2.1: Example van de Vooren geometries for $l = 1$ and a as specified in Eqn. 2.4

Notably the leading edge corresponds to $\theta = \pi$ and is located at

$$z = -a\frac{2^k}{(1 + \epsilon)^{k-1}} + l \tag{2.3}$$

Thus setting $l = 1$ and

$$a = \frac{(1 + \epsilon)^{k-1}}{2^k} \tag{2.4}$$

positions the leading edge at location (0,0) in the z plane. Furthermore, $\theta = 0$ specifies the trailing edge and for $l = 1$ corresponds to location (1,0) in the z plane [22]. Under these specifications an overall chord length of 1 is obtained and two such van de Vooren airfoils are presented in Figure 2.1. This specification of the chord length is needed for the discretisation of the airfoil surface in panels as implemented in the 2D potential flow

solver, which is described in the following section.

2.2.1.2 CST parametrisation

A non-symmetric airfoil can be obtained by the so called "class function / shape function" transformation technique (CST) [24], in which the upper and lower surfaces of the airfoil are parametrised separately by a function of the form:

$$\frac{y}{c} = C\Big(\frac{x}{c}\Big)S\Big(\frac{x}{c}\Big) + \frac{x}{c}\frac{\Delta z_{te}}{c} \tag{2.5}$$

where c denotes chord length, Δz_{te} represents trailing edge thickness and $S\Big(\frac{x}{c}\Big)$ represents the shape function.

The class function $C\Big(\frac{x}{c}\Big)$ is given by

$$C\Big(\frac{x}{c}\Big) = \Big(\frac{x}{c}\Big)^{N_1}\Big[1 - \frac{x}{c}\Big]^{N_2} \tag{2.6}$$

$$\text{for} \qquad 0 \leq \frac{x}{c} \leq 1$$

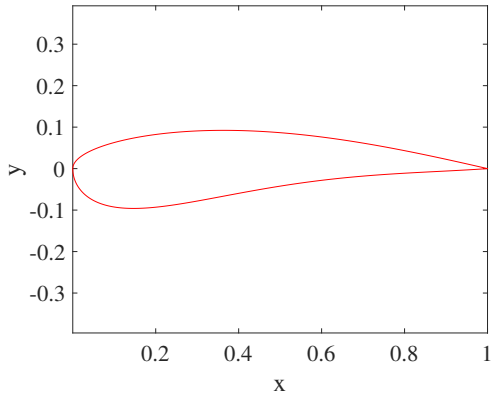
The exponents N_1 and N_2 define the general category of shapes to be represented [25]. An airfoil geometry can be obtained by setting $N_1 = 0.5$ and $N_2 = 1$. For these values the term $\sqrt{x/c}$ enforces a round leading edge and the factor $\Big(1 - \frac{x}{c}\Big)$ yields a sharp the trailing edge [24]. For the shape function $S\Big(\frac{x}{c}\Big)$ Kulfan and Bussoletti [24] have suggested the use of a weighted sum of Bernstein binomials as follows:

$$S\Big(\frac{x}{c}\Big) = \sum_{i=0}^{n}\Big[w_i \cdot K_{i,n} \cdot \Big(\frac{x}{c}\Big)^i \cdot \Big(1 - \frac{x}{c}\Big)^{n-i}\Big] \tag{2.7}$$

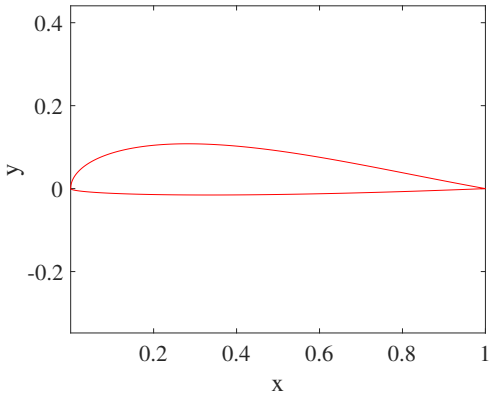
where

$$K_{i,n} = \frac{n!}{i!(n-1)!} \tag{2.8}$$

The geometry is thus defined by the selection of weights w_i, which can be collected in a vector representation as follows $\mathbf{w} = [w_1,\ w_2,\ \ldots.\ w_n]^T$ for $i = 1, n$. A couple of examples are presented in Figure 2.2.



(a) Airfoil section with
$\mathbf{w} = [-0.4,\ -0.004,\ -0.07,\ 0.21,\ 0.27,\ 0.23]^T$



(b) Airfoil section with
$\mathbf{w} = [-0.04,\ -0.04,\ -0.04,\ 0.31,\ 0.27,\ 0.18]^T$

Figure 2.2: Example CST geometries for $N_1 = 0.5$ and $N_2 = 1$

The Bernstein polynomials guarantee the existence of two roots at x=0 and x=1 and are positive in the interval in between. This combined with the featured provided by the class function in terms of round leading edge and sharp trailing edge results in an airfoil geometry, which is defined separately for the upper and lower surface by Equation 2.6 [25]. This geometric representation allows for direct control the leading edge radius and the boattail angle β as follows:

$$S(0) = \sqrt{\frac{2R_{le}}{c}} \qquad\qquad S(1) = \tan\beta + \frac{\Delta z_{te}}{c} \tag{2.9}$$

Returning to the optimisation framework under discussion, a priori definition of the shape by setting $N_1 = 0.5$ and $N_2 = 1$, results in a fixed class function through all design cycles with shape changes enforced by changes in the weightings of $S\left(\frac{x}{c}\right)$. Furthermore, the use of weighted factors allows for the dimension n of the input parameter vector to be selected for the particular application. This results in flexibility of design space definition and thus presents opportunities for framework testing in terms of time and memory complexity.

2.2.2 2D potential flow solver

The solver incorporates a two-dimensional second order vortex panel method. Using linear interpolation the surface is approximated by np number of vortex panels with linearly varying strength as presented in Figure 2.3

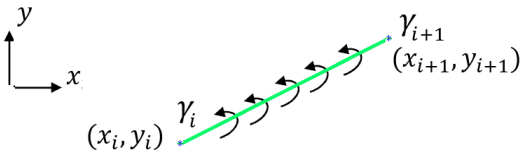


Figure 2.3: A single panel notation

The vortex strength at each panel edge is denoted as γ_i, with $i = 1$ and $i = np + 1$ representing the trailing edge. For this project the panel number np was kept constant at 200. Noting that the trailing edge point is repeated, there are $np+1$ edge coordinate pairs (x_i, y_i) and thus $np+1$ unknown vortex strengths need to be obtained. The satisfaction of the Kutta condition requires that the vortex sheet strength at the trailing edge γ_1 is zero. Furthermore,as the first and final point repeat, it is known that $\gamma_1 = \gamma_{np+1}$.

The rest of the vortex strengths can be obtained from a system of linear equations derived by considering the contributions to the stream function as presented by [17] and summarised in this section. At each coordinate pair, the stream function is a superposition of the influence of the free stream and that of all panels. Thus

$$\psi^{overall}(x_i,\, y_i) = \psi^{panels}(x_i,\, y_i) + \psi^{free\,stream}(x_i,\, y_i) \tag{2.10}$$

The contribution from each panel (i) can be denoted as $(\gamma_i f_i + \gamma_{i+1} f_{i+1})$ where f_a and f_b are known as the influence coefficients and their derivation is presented in Appendix A. Notably, they depend entirely on the coordinates of the panel. The combined contribution from all panels to the stream function at one of the panel edges $(x_i,\, y_i)$ can be expressed as follows:

$$\psi^{panels}(x_i,\, y_i) = \sum_{p=1}^{np} [\gamma_p f_p + \gamma_{p+1} f_{p+1}] \tag{2.11}$$

To deal with the conditions at γ_1 and γ_{np+1} described above, Equation 2.11 can be rewritten as

$$\psi^{panels}(x_i,\, y_i) = \sum_{j=1}^{np+1} [F]_{i,j}\, \gamma_j \tag{2.12}$$

where

$$[F]_{i,j} = \begin{cases} f_i^j & j = 1 \\ f_i^j + f_{i+1}^{j-1} & j = 2, 3, ...np \\ f_{i+1}^{j-1} & j = np + 1 \end{cases} \tag{2.13}$$

Thus the contribution from each panel i to the stream function is only a function of the coordinates of its edges and the strength of the vortex strength there (γ_i). Furthermore, the free stream contribution to the stream function depends only on the angle of attack α, which itself depends on the positioning of the airfoil and thus directly on the panel edge coordinates as well.

$$\psi^{free\,stream}(x_i,\, y_i) = y_i cos(\alpha) - x_i sin(\alpha) \tag{2.14}$$

Returning to Equation 2.10, the airfoil surface is a streamline which means that a boundary condition must be applied keeping $\Psi_{overall}$ constant from one panel edge to the next.

$$\psi^{overall}(x_{i+1},\, y_{i+1}) - \psi^{overall}(x_i,\, y_i) = 0, \tag{2.15}$$

$$\psi^{panel}(x_{i+1},\, y_{i+1}) - \psi^{panel}(x_i,\, y_i) = \psi^{free\,stream}(x_i,\, y_i) - \psi^{free\,stream}(x_{i+1},\, y_{i+1}) \tag{2.16}$$

For the purposes of the analysis and the following discussion, the left hand side of equation 2.16 can be simplified using equation 2.12 as $A\gamma$ where matrix A $(np+1 \times np+1)$ contains the dependence of the panel contribution on the edge coordinates and γ is a column vector $(np+1 \times 1)$ of vortex sheet strength at each location $(x_i,\, y_i)$. Denoting the contribution from the free free stream as a column vector $\mathbf{b}$ $(np+1 \times 1)$, Equation 2.16 can be rewritten in the following matrix form

$$A\gamma = \mathbf{b} \tag{2.17}$$

and solved using methods for solving linear systems of equations. Notably matrix A and vector $\mathbf{b}$ depend on the panel edges locations and thus their sensitivity to perturbations in the geometry can be examined. Furthermore, this equation can result in the calculation of the vector of vortex sheet strengths at the panel edges γ and thus the surface pressure distribution as follows:

$$c_{p,i} = 1 - \gamma_i \tag{2.18}$$

where $c_{p,i}$ denotes the pressure coefficients calculated at each panel edge location $(x_i,\, y_i)$. Additionally, integrating the γ_i distribution over the surface of the airfoil results in the calculation of the overall circulation Γ.

2.2.3 Integral boundary layer solver

Having obtained a solution of the inviscid region the evolution of two boundary layers is modelled, one located on the upper and one on the lower surface, each starting from the leading edge stagnation point. This point is determined to be at the location at which the sign if γ_i changes. The velocity at the edge of the boundary layers is estimated from pressure distribution provided by the potential flow solver and used to model the boundary layer evolution from an initially laminar state.

Laminar regime: For laminar flow the solver implements Thwaites' solution of the momentum integral equation and includes an empirical test for transition proposed by Eppler and Somers [26] and a test for separation based on the Thwaites' method [17]. The governing flow equations, together with the empirical criteria for transition, separation and reattachment are presented in Appendix B.

Turbulent regime: In the case of transition to turbulence the code implements the momentum and kinetic energy integral equations for a turbulent boundary layer and uses an empirical threshold value for the energy shape factor, below which a turbulent boundary

layer will be separated, as suggested by Eppler and Somers [26]. The governing equations and additional clarifications are presented in Appendix C.

For the purpose of their numerical implementation and the calculation of the momentum thickness θ and the kinetic energy thickness δ_E, the governing equations can be rewritten as a system of differential equations:

$$\frac{d\mathbf{y}}{dx} = f(x, \mathbf{y}) \tag{2.19}$$

where $\mathbf{y}$ is defined

$$\mathbf{y} = \begin{bmatrix} \theta \\ \delta_E \end{bmatrix} \tag{2.20}$$

This system can readily be solved with differential equation solvers such as *ode*45 in MATLAB and or the ODEPACK collection of Fortran 77 solvers, which can be implemented in Fortran 90. These libraries however do not allow for transparent line by line source code differentiation. For this reason two more simplistic hand-written numerical solvers were implemented based on the Euler explicit method and the more accurate explicit Fourth Order Runge-Kutta Scheme. Both solvers are present in the final code architecture, however the latter one was used to obtain all the results presented in Chapter 3. This was due to the better accuracy of its results in comparison to the Euler implementation when compared to the solutions obtained with the more sophisticated ode45 MATLAB routine.

Calculation of the lift and drag coefficients From the panel method the circulation per panel can be calculated as $\Delta(\gamma_i+\gamma_{i+1})/2$, where Δ is the length of the panel. Summing up over all the panels to obtain the overall circulation Γ and using the Kutta-Joukowsky theorem we can obtain the lift coefficient as

$$C_L = -2\frac{\Gamma}{Uc} \tag{2.21}$$

where c is the aerofoil chord [17].

Additionally, the trailing edge momentum thickness θ_{te} and displacement thickness δ^*_{te} can be extracted from the upper and lower surface boundary layer calculation as:

$$\theta_{te} = \theta_{te,lower} + \theta_{te,upper} \qquad \delta^*_{te} = \delta^*_{te,lower} + \delta^*_{te,upper} \tag{2.22}$$

Using these values the momentum thickness at infinity θ_∞ can be obtained through the Squire-Young relation as follows:

$$\theta_\infty = \theta_{te}\left[\frac{u_{e,te}}{U}\right]^{(H_{te}+5)/2} \tag{2.23}$$

where u_e^{te} is the trailing-edge velocity [17]. This value can then be used for the estimation

of the drag coefficient as as shown by [27] and [17]:

$$C_D = 2\frac{\theta_\infty}{c} \tag{2.24}$$

Finally, equations 2.21 and 2.24 can be divided to obtain the ratio of lift-to-drag coefficients or equivalently lift-to-drag, which will be the main airfoil characteristic investigated by the optimisation algorithm presented in Section 2.4.

2.3 Gradient estimation

Once the lift-to-drag ratio was obtained, the next step was to compute its gradient with respect to several input parameters specified by the choice of geometry definition. Three different methods were implemented separately for the computation of the partial derivatives forming the gradient and their performance is compared in Section 3.1.1. The first method uses the a forward difference formulation (Equation 1.14) with a pre-selected step size. The other two methods are based on automatic differentiation in forward and reverse mode respectively. For their implementation, a transformation based AD method was chosen, which was realised using both hand-written differentiation and an online source-to-source transformation tool called TAPENADE.

2.3.1 TAPENADE

This automatic differentiation engine was developed by Hacoët and Pascual at INRIA Sophia Antipolis [19, 20]. TAPENADE takes source code as input and by defining a top subroutine, dependent and independent variables the user can download a differentiated version of their code, in forward or reverse mode. The tool is specialised for the creation of an efficient adjoint code, incorporating control- and data-flow reversal and a Store-All approach with some recompilation and checkpointing [19]. Current implementations support the languages Fortran 77, Fortran 90 and C. It is available both online and as a downloadable program, however in this project only the online version was used. This decision resulted in issues due to prolonged periods of server downtime without prior notification, leading to careful scheduling of tasks to avoid delay of the project.

2.3.2 Challenging aspects

Two particularly challenging aspects of implementing forward or reverse mode AD were met during the development of the solvers. Although, successfully overcome,they highlight a difficult aspect of using TAPENADE and a change to the base flow solver.

External libraries: The flow solver uses the LAPACK Fortran 90 library [21] to manipulate large matrices and to find solutions to systems of simultaneous equations such

as 2.17. TAPENADE treats these routines as "black-boxes" and is unable to differentiate them [11]. The developer is thus required to separate the code in parts, differentiating them separately and then "connect" them by a hand-written differentiation of the "black-box" routines. In the case of Equation 2.17, the hand-written routines are derived by [28] and stated as follows in the notation presented in Section 1.4.3:

$$\gamma = A^{-1}\mathbf{b} \tag{2.25}$$

In forward mode:

$$\dot{\gamma} = -A^{-1}\dot{A}A^{-1}\mathbf{b} + A^{-1}\dot{\mathbf{b}} \tag{2.26}$$

In reverse mode:

$$\bar{\mathbf{b}} = (A^{-1})^T\bar{\gamma} \tag{2.27}$$

$$\bar{A} = -\bar{\mathbf{b}}\gamma^T \tag{2.28}$$

Control statements: As described previously the boundary layer solver uses empirical criteria for separation, transition and reattachment of the flow. These were initially implemented using "if-else" statements. Due to the use of line by line source code differentiation, all of these statements had to be replaced with alternative differentiable switch functions to enable the chain rule of differentiation. The new control flow makes use of the monotonic and differentiable hyperbolic tangent functions to construct logical statements.

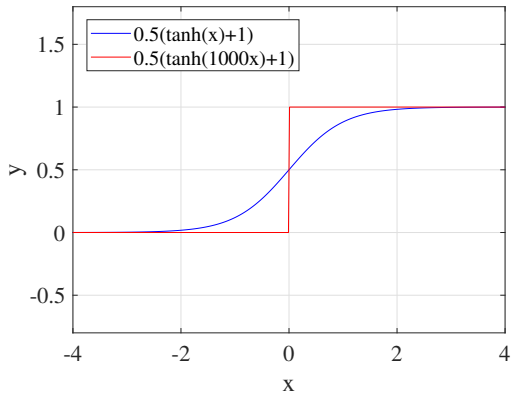


Figure 2.4: Shape of the scaled *tanh* function used to construct the logical statements

For example if laminar separation occurs, we aim to calculate the momentum thickness θ and the energy thickness δ_E to check for possible reattachment as outlined in Appendix B. Previously, an "if-else" structure was used to check whether H_E has exceeded a threshold value $H_{E,th} = 1.58$ to indicate reattachment. Following the check the appropriate code was executed to compute θ and δ_e for attached or separated flow. Alternatively, the new

implemented control statement acts as a logic gate:

$$\theta = (a)\theta_{attached} + (1 - a)\theta_{separated} \tag{2.29}$$

where

$$a = \frac{1}{2}(tanh(1000(H_E - H_{E,th}) + 1) \tag{2.30}$$

The factor of $\frac{1}{2}$ scales the $tanh$ function between values of 0 and 1 and the factor 1000 allows a steep slope around the threshold value, which enables a quick switch at the threshold. From the shape of the $tanh$ function presented in Figure 2.4 we can see that for values of H_E below the threshold, $a = 0$ and thus Equation 2.29 becomes $\theta = \theta_{separated}$. Conversely, for values above the threshold, $a = 1$ and $\theta = \theta_{attached}$ due to reattachment. δ_E can be computed in an analogous way. Similar control statements were created for all empirical checks listed in Appendix B and Appendix C.

Notably, the new control statement introduces an additional calculation as the parameters have to be calculated for both flow states. This is small, however, in comparison to the the potential run time savings enabled by the use of reverse AD, which will be investigated in Section 3.2.

2.3.3 Developed Solvers

Implementing both FDM and AD resulted in the completion of four different solvers, between which the user can choose depending on the application. This, coupled with the user's choice of geometry definition, allows for great flexibility in design space selection and for the investigation of the performance of both FDM and AD within the framework of numerical shape optimisation.

The first solver includes the full flow solver (geometry definition, 2D potential flow solver and integral boundary layer solver) and results in the computation of the lift-to-drag ratio only, which will be the objective function f in the optimisation framework presented in the following section. In this report this solver will be referred to as the "original solver".

The second solver, implements the finite difference method (FDM) for the computation of the gradient of the objective function with respect to the input parameters, defining the arifoil geometry. To achieve this the "FDM solver" incorporates the original solver with an extension for derivative calculation using a forward difference formulation (Equation 1.14). Depending on the application, a study of the most appropriate perturbation step-size is necessary. The selection for this project is presented in Section 3.2 separately for all used geometry parametrisations.

The third solver relies on forward mode automatic differentiation (FAD) for the the computation of the gradient of the objective function with respect to the input parameters. In

this solver each line of the original solver is differentiated. Therefore the objective function and all intermediate values are calculated parallel to their differential with respect to a single input parameter. As a result this "FAD solver" must be run once for changes in each input parameter individually in order for the full gradient vector to be formed.

In contrast, the computational cost of evaluating the gradient with the fourth solver is independent of the number of input parameters. This solver includes the original solver and a separate code structure of differentiated routines in the manner required by reverse automatic differentiation. This "RAD solver" requires a single forward run of the original solver to compute the objective function and a single run of the differentiated code resulting in a computation of all partial derivatives forming the gradient simultaneously. Therefore the RAD solver has a computational cost for evaluating the gradient of a small multiple of the cost for evaluating the objective function itself, independent of the number of input parameters. Its internal structure is presented in the figure below.

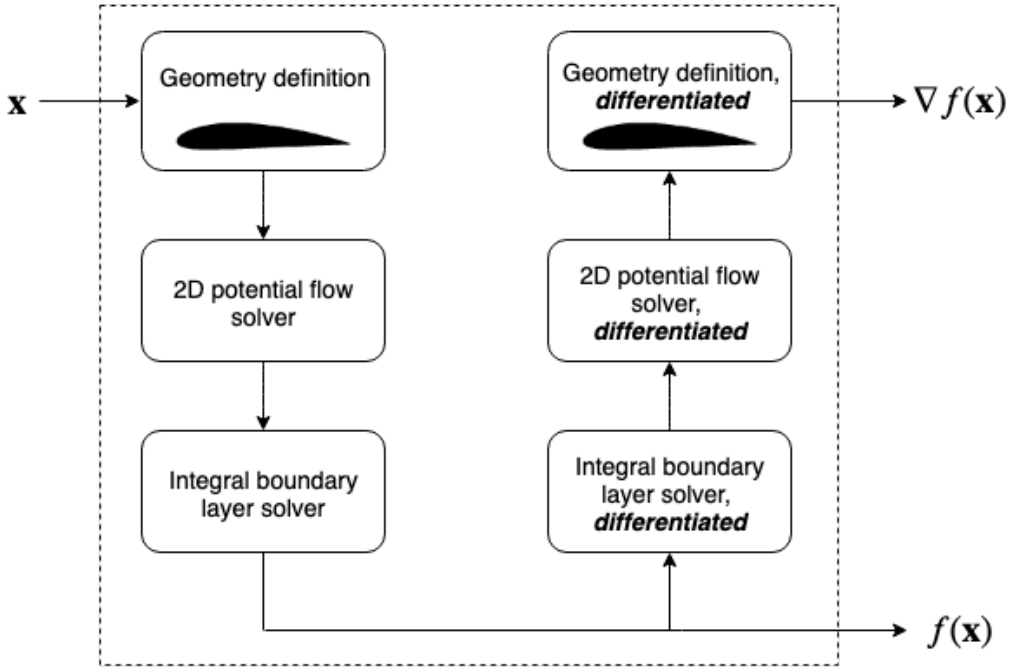


Figure 2.5: Internal structure and data flow within the RAD solver

The process of objective function and gradient evaluation with the RAD solver is as follows:

1. The vector of input parameters $\mathbf{x}$ is used to define the shape of the airfoil.
2. The 2D potential flow solver then calculates the surface pressure coefficients, the location of the stagnation point and the overall circulation leading to an estimation of the lift coefficient C_L.
3. The boundary layer solver is initialised leading to identifications of points of laminar to turbulent transition, flow separation and reattachment. The solver results in an estimation of the drag coefficient C_D and the ratio (C_L/C_D) , which is the objective

function f being maximised by the chosen optimisation algorithm as will be outlined in Section 2.4.

4. Following this forward run, the RAD solver seeds a change in the objective function $\bar{f} = 1$. The code then enters the differentiated routines of items $1 - 3$ in reverse fashion characteristic for reverse AD, obtaining the gradient of the objective function with respect to all input parameters.

2.4 Optimisation

2.4.1 Problem statement

Following the development of the four solvers, we investigated their implementation to the problem of numerical airfoil shape optimisation. In the optimisation process we aim to obtain the maximum lift-to-drag ratio achievable under the imposed flow model and under the constraints on the shape as defined from the parametrisation method.

Setting the objective function f as the lift-to-drag ratio the problem statement is thus given by:

$$\max_{\mathbf{x}} \frac{Cl}{Cd} \tag{2.31}$$

$$\mathbf{a} \leq \mathbf{x} \leq \mathbf{b}$$

where $\mathbf{x}$ is a vector comprised of the input parameters defining the geometry of the airfoil under the chosen parametrisation method. $\mathbf{a}$ and $\mathbf{b}$ are vector functions describing the constraints placed on $\mathbf{x}$.

In the case of van der Vooren parametrisation, conformal mapping ensures that within the bounds of the specific mapping the resultant shape will be airfoil-like. Under the CST parametrisation the same constraint in the shape is achieved by setting $N_1 = 0.5$ and $N_2 = 1$ and thus fixing the class function defining the underlying geometry. This limitation in the design space is justified by the knowledge of the engineer that airfoil shapes achieve highest ratios of lift-to-drag. No optimisation was done under direct geometry definition by coordinates as no guarantee of sensible geometry was present.

2.4.2 Unconstrained optimisation

Initially the optimisation was started without additional bounds in order to explore the full design space delineated by the van der Vooren mapping. This was achieved by setting each entry in vectors $\mathbf{a}$ and $\mathbf{b}$ to infinity in the formulation above. The design choice was made to use the BFGS method described in Section 1.3. The algorithm was implemented from scratch in Fortran 90, however no sensible results were obtained as the optimisation quickly

settled for negative values of ϵ or extremely large value k. Although these parameters truly achieve larger values of the objective function from the flow solver, they result in physically unrealistic shapes and cause termination of the optimisation due to fatal errors in the flow solver.

2.4.3 Constrained optimisation

2.4.3.1 In two dimensions

A two dimensional design space was delineated by the implementation of van de Vooren mapping with a set chord length of one, as described in section 2.2.1.1. Therefore the following vector of input parameters was defined:

$$\mathbf{x} = \begin{bmatrix} k \\ \epsilon \end{bmatrix} \tag{2.32}$$

Although unsuccessful, the unconstrained optimisation case showed that box constraints on the input parameter vector are necessary. Based on the definition of the input parameters by the conformal mapping method (Section 2.2.1.1) the following constraints were incorporated:

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix} \leq \begin{bmatrix} k \\ \epsilon \end{bmatrix} \leq \begin{bmatrix} 2 \\ 1 \end{bmatrix} \tag{2.33}$$

All combinations of the limiting values of ϵ and k results in a circle and all other combinations of allowed values result in standard airfoil geometries.

2.4.3.2 In n dimensions

A n-dimensional design space can be defined by the implementation of the CST parametrisation. The input parameter vector in this case is the vector of weights $\mathbf{w}$, as defined in Section 2.2.1.2 and subject to the following constraint:

$$\begin{bmatrix} -1 \\ -1 \\ : \\ 0.001 \\ 0.001 \end{bmatrix} \leq \begin{bmatrix} w_1 \\ w_2 \\ : \\ w_{n-1} \\ w_n \end{bmatrix} \leq \begin{bmatrix} -0.001 \\ -0.001 \\ : \\ 1 \\ 1 \end{bmatrix} \tag{2.34}$$

Due to the use of Bernstein binomials for the shape function of the upper and lower surface, which are positive in this range, parameters with negative values control the shape of the lower surface and those with positive values the upper. The limit of ± 0.001 eliminates the possibility of a weight vector with all zero elements which corresponds to a line shape. In

our investigation, which will be presented in Section 3.2 we selected two test design spaces, one with $n = 6, \mathbf{x} \in R^6$ and another with $n = 10, \mathbf{x} \in R^{10}$.

2.4.3.3 The numerical optimisation algorithm

For the case of box constraints, the new optimisation algorithm implemented was a modification of "Algorithm 778: L-BFGS-B" aimed at large scale bound constrained optimisation [29]. This method improves upon the basic BFGS implementation in two main aspects: memory requirements and possibility of handling of bound constraints on the input variables.

This method is an extension of the limited-memory BFGS algorithm (L-BFGS) and as such belongs to the family of limited-memory quasi-Newton methods. Given an input vector comprised of n parameters, these algorithms store the approximation of a Hessian implicitly through only a few vectors of length n, rather than the full $(n \times n)$ approximation as in the standard quasi-Newton methodologies, where n is the number of input parameters. Despite the reduced storage requirements, these algorithms do not demonstrate rapid convergence [5]. The L-BFGS-B method, itself shows both these characteristics. It further extends upon the L-BFGS algorithm by its ability to implement bound constraints on the input variables, which is considered to be its main advantage [29].

A short summary of the algorithm is as follows. At each iteration a limited-memory BFGS approximation to the Hessian is computed using gradient information provided by the solver and stored in a compact form that is efficient for bound constrained optimisation problems [29]. Using this approximation a quadratic model of the objective function is created and a search direction is computed using a projected gradient method which is capable of identifying which variables have reached their constraints [29]. The quadratic model is then reiterated based on the remaining variables and a line search is performed along the search direction. Several termination conditions can be implemented, with the one used in this project relying on the the norm of the projected gradient $\|proj\ g\|_\infty$ becoming smaller than a threshold value tol set by the user.

The Fortran 77 and Fortran 90 implementations of the algorithm are available freely online, however for the purpose of this project additional drivers were created to to enable gradient estimation through forward and reverse automatic differentiation and thus incorporate both the RAD and FAD developed in this project.

2.5 Full framework overview

Th full framework for numerical optimisation of 2D airfoil sections developed in this project was completed by the implementation of the RAD, FAD and the FDM solver within the L-BFGS-B algorithm for the computation of the objective function and its gradient with

respect to the input parameters. A flow chart of the overarching optimisation process for the implementation of the RAD solver is presented in Figure 2.6. The bound constrained optimisation proceeds as follows:

1. A search direction d_i is computed based on the the projected gradient method, using the RAD solver to obtain information about the objective function f_i and its gradient ∇f_i at the current position of the input parameter vector x_i.

2. A line search, subject to the bounds on x, is performed along d_i to obtain a step of length λ_i resulting in a new position vector $x_{i+1} = x_i + \lambda_i d_i$.

3. At the new position x_{i+1} the RAD solver is used to compute f_{i+1} and ∇f_{i+1} and the quadratic model of the function is updated.

4. A check of convergence is carried out based on the the norm of the projected gradient $\|projg\|_\infty$ becoming smaller than a threshold value tol. If the value hasn't converged, a new iteration is initialised starting from step 1 with initial parameter vector of x_{i+1}. If convergence is achieved, the process terminates outputting x_{i+1} as the position x^* of the optimal value of the objective function f^* equal to f_{i+1}.

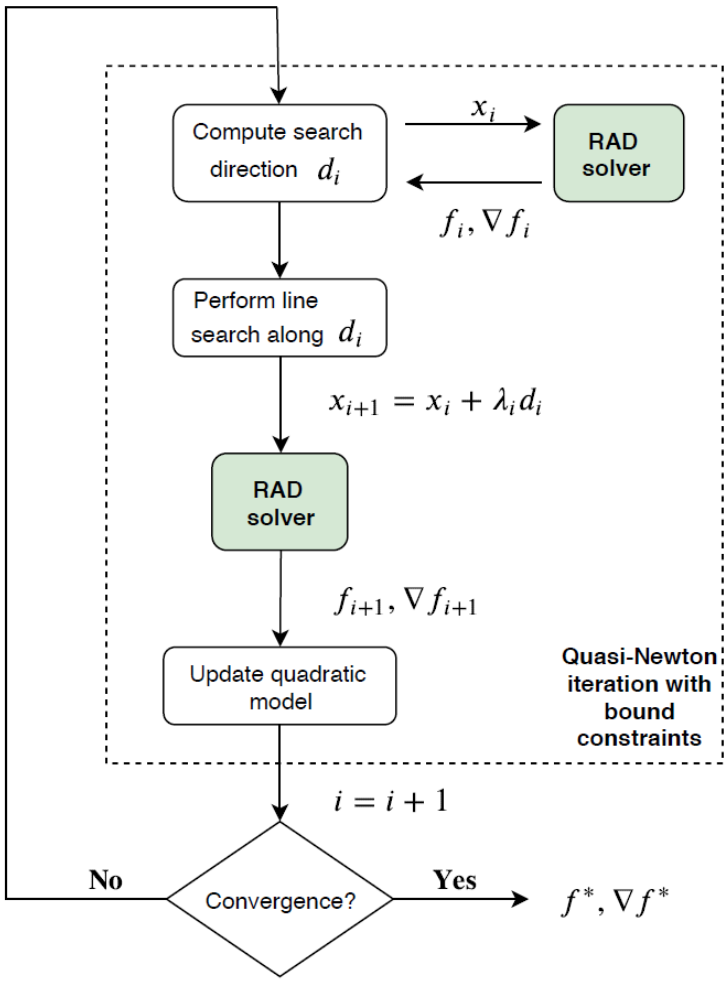


Figure 2.6: Flow chart of the full optimisation process

Chapter 3

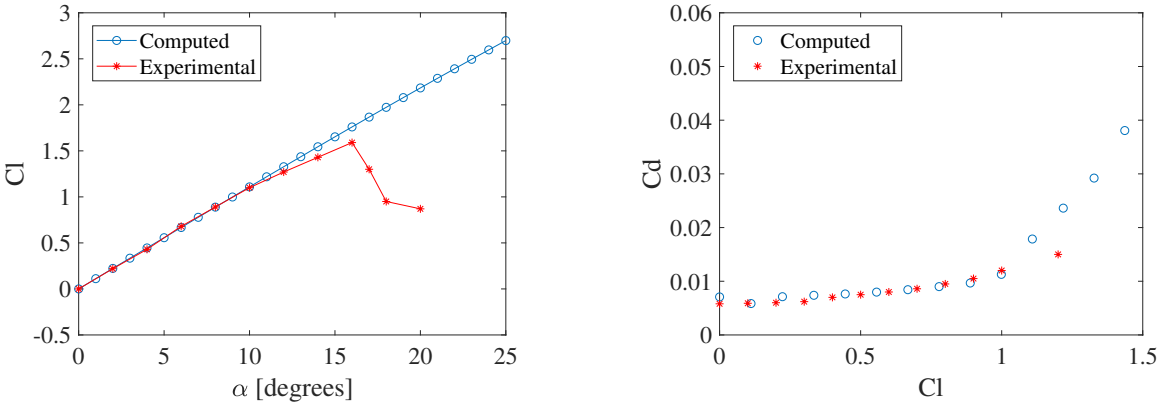
Results and discussion

This chapter presents a thorough investigation of the developed framework for shape optimisation of 2D wing sections. It outlines the case studies conducted to validate and compare the performance of all four solvers developed in this project and concludes with an investigation of their realisation within the L-BFGS-B algorithm applied to the optimisation of the lift-to-drag ratio with respect to the airfoil's geometry parameters.

The results are separated in three different categories. First, a validation of the original flow solver is presented and its limitations are discussed. This investigation is followed by three test case studies of the accuracy with which automatic differentiation computes the first derivatives of different objective functions with respect to perturbation of the input geometry parameters. The final section presents and compares the performance of the FAD, RAD and FDM solvers in the context of bound constrained optimisation for three different design spaces defined by two, six and ten input parameters. It must be noted that all presented time tests were carried out on a single 2.5 GHz CPU virtual machine with 2048MB RAM running an Ubuntu operating system.

3.1 Flow solver validation

The NACA 0012 airfoil section [30] was selected for the validation of the original flow solver. Data obtained by wind-tunel testing, presented in [31], was compared to the computed results from the flow solver and presented in Figure 3.1.



(a) C_L at different angles of attack α

(b) C_D for different values of C_L

Figure 3.1: Comparison between experimentally measured data and computed estimates for NACA 0012 at $Re = 3 \times 10^6$

At small angles of attack the computed values agree strongly with the experimental results. As the angle of attack increases, the computed lift coefficient deviates more from the

experimental values, which decrease due to viscous effects. As the angle of attack increases, the adverse pressure gradient rises causing the boundary layer to separate. This tends to reduce the velocities along the upper surface of the airfoil causing a reduction in the lift coefficient, a phenomenon known as stall. Thus the inviscid panel method solution, implemented by the flow solver, is unable to predict the near-stall behaviour. For this reason the measurements and optimisation results presented in this chapter are limited to angles between 0° and 10°.

The solution to this problem lies within the method of coupling between the inviscid and the viscous solution. Currently the solver separates the flow field in an inviscid region and viscous boundary layer. The pressure coefficients at the edge of the boundary layer at each panel edge are calculated using the 2D panel method and then used as an input to the integral boundary layer solver.

The most simple method of introducing more accurate coupling between the boundary layer and the inviscid solution is to introduce iteration in the integral boundary layer solver as suggested by [15]. Once the external flow is calculated, the pressure coefficients at each panel edge are used to obtain an initial approximation of the boundary layer state and indicate whether transition, separation and reattachment have occured. In the case of a separation bubble, the pressure coefficient is set to a constant value between the separation and reattachment location. In the case of turbulent separation, a wake can be modelled by setting the pressure coefficients beyond the separation point to the free stream value. The new pressure distribution can be used to iterate the boundary layer solution and this procedure can be repeated until convergence is reached between two subsequent pressure distributions [15]. Alternative advanced methods for coupling between the potential solution and the integral boundary layer equations are summarised by [32].

3.1.1 Sensitivity analysis through forward mode AD

Following the flow solver validation, we proceeded by comparing the the accuracy with which each of the solvers computed the first derivatives of different objective functions with respect to the input geometry parameters. This was achieved through three test cases: sensitivity analysis of both contributions to the stream function (from the panels and the free stream) to changes in the panel edge coordinates and direct comparison of the partial derivatives of the lift-to-drag ratio with respect to the two parameters defining the van der Vooren mapping for the FDM, RAD and FAD solver.

3.1.1.1 Case 1:

Taking all 402 panel edge coordinates as input parameters we first performed a sensitivity study using the free stream contribution to the stream function (Equation 2.14) and thus

examined the sensitivity of vector **b** with respect to perturbations in each parameter individually. In this case the function was simple, the exact derivative was easily calculated by hand and compared to the results of the the FAD solver. The values showed perfect match validating the performance of this solver and and demonstrate its accuracy in computing the first derivatives.

3.1.1.2 Case 2:

Following this result a further investigation was set up to investigate the performance of the FDM solver in terms of derivative approximations in comparison to the validated FAD solver. This was done through sensitivity analysis of the panel contribution to the free stream function, represented in matrix A (Equation 2.17) with respect to all 402 control parameters defined above. The investigation proceeded as follows:

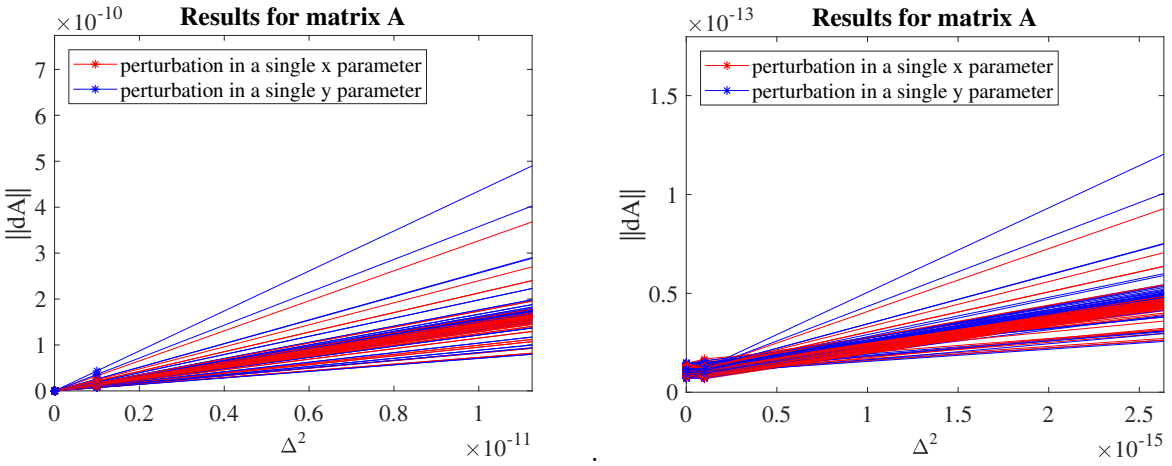
- **Step 1:** Matrix A was calculated for the given initial control parameters using the FDM solver and recorded as $A_{original}$. Taking one of the 402 control parameters (p) and perturbing its value with a step-size Δ resulted in revaluation of matrix A (A_{new}) and estimation of the change due to the perturbation $dA_{FDM} = A_{new} - A_{original}$.

- **Step 2:** Using the FAD solver, the forward mode AD was implemented by setting the change in the same parameter control parameter p to one ($\dot{p} = 1$), a process known as seeding. A run of the differentiated solver resulted in the corresponding first-order change in matrix A recorded as dA_{AD} and used to estimate $dA_{AD} = \frac{dA_{AD}}{dp}\Delta$.

- **Step 3:** The change in the estimates from FDM and forward mode AD was recorded in the matrix $dA = dA_{FDM} - dA_{AD}$

- Steps one to three were then repeated for a range of step sizes from 10^{-9} to 10^{-4} in multiples of ten. The whole process was then reiterated for all 402 control parameters individually with results presented in Figure 3.2.

  As expected the difference in the predictions from the FDM and FAD solvers reduces with decreasing Δ due to a decreasing the truncation error in the derivative approximation of the finite different method. Additionally, Figure 3.2b also demonstrates that there is an optimum size of Δ below which a round-off error is introduced as discussed in Section 1.4.1. Thus in this case the error in the derivative approximation of the FDM solver will be minimum for $\Delta = 10^{-8}$.



(a) Results for step sizes from 10^{-9} to 10^{-5} (b) Zoomed in results in the vicinity of $\Delta = 10^{-9}$

Figure 3.2: Comparison of the error in first derivative computation resulting from the FDM and FAD solvers for a range of step sizes for all 402 control parameters. The largest values with $\Delta = 10^{-4}$ have been cut off from the figure to allow for the straight line relation to be shown

3.1.1.3 Case 3:

The final sensitivity study was conducted as a preliminary investigation of the sensitivity of lift-to-drag ratio to changes in the parameters defining the aifoil's geometry under the van der Vooren mapping. The results are summarised in the following table:

$k = 1.95$, $\epsilon = 0.03$					$k = 1.2$, $\epsilon = 0.7$				
	Forward AD	Reverse AD	Finite difference			Forward AD	Reverse AD	Finite difference	
				Δ					Δ
$\frac{\partial f}{\partial k}$	160.28	160.28	155.78	10^{-3}	$\frac{\partial f}{\partial k}$	17.35	17.35	17.38	10^{-3}
			158.90	10^{-4}				17.85	10^{-4}
			162.82	10^{-5}				24.85	10^{-5}
			501.94	10^{-6}				47.98	10^{-6}
			2072.75	10^{-7}				501.40	10^{-7}
			4302.78	10^{-8}				233.76	10^{-8}
$\frac{\partial f}{\partial \epsilon}$	-241.42	-241.42	-255.02	10^{-3}	$\frac{\partial f}{\partial \epsilon}$	-6.45	-6.45	-6.41	10^{-3}
			-242.28	10^{-4}				-6.42	10^{-4}
			-222.10	10^{-5}				-1.93	10^{-5}
			-590.98	10^{-6}				31.92	10^{-6}
			3306.32	10^{-7}				283.23	10^{-7}
			1672.04	10^{-8}				-294.52	10^{-8}

Table 3.1: Sensitivity analysis of the lift-to-drag ratio f with respect to k and ϵ conducted with both the FAD, RAD and FDM solvers for a range of step sizes Δ

The smaller number of input parameters in this case allows us to see more clearly the great dependency of the FDM derivative approximation on the perturbation step-size Δ. This highlights the need for a step-size selection before the implementation of the FDM solver. Additionally, the table includes results from the BAD solver in order to show the agreement between the FAD and RAD solvers in the first derivative estimation.

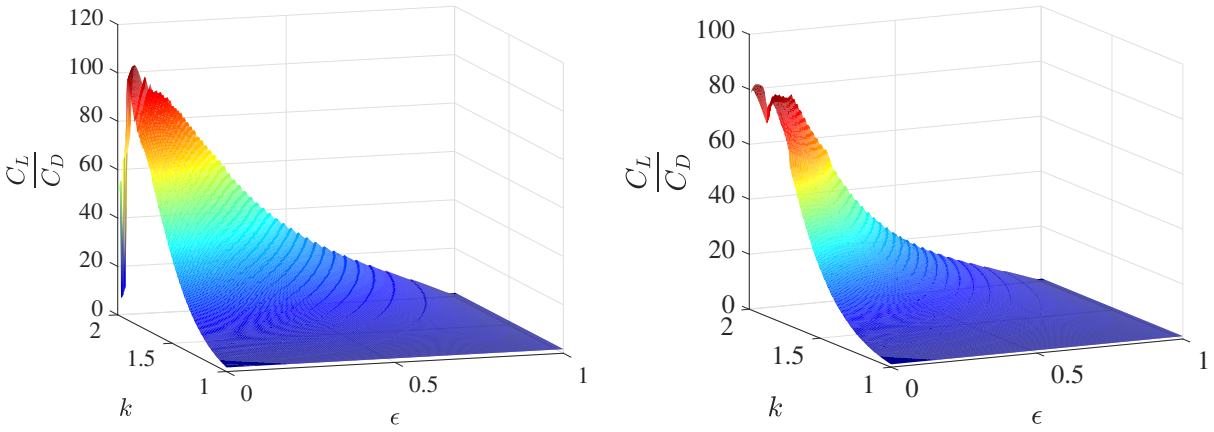
Based on this results, a step size of 10^{-4} is chosen for the application of the FDM solver in the calculation of the partial derivatives of the lift to drag ratio with respect to input parameters defined by the van de Vooren mapping.

3.2 Optimisation

Following the successful validation of the original solver and the selection of perturbation step size for the FDM solver, we addressed the optimisation problem formulated in Section 2.4.1. The optimisation investigation was conducted by implementing each of the solvers individually within the L-BFGS-B framework and comparing their performance in three different design spaces, defined by a two-, six- or ten-dimensional input vector respectively using one of the two parametrisation methods available (van de Vooren mapping and the CST technique).

3.2.1 Optimisation in two dimensions

Initially box constrained optimisation was carried out using the RAD solver in a design space presented by the van der Vooren mapping under constraints on the input parameters presented in Section 2.4.2. For this purpose a Reynolds number of 3×10^6 and angles of attack of 5° and 10° were selected based on the accuracy limits of the flow solver as discussed in Section 3.1.



(a) $Re = 3 \times 10^6$ and $\alpha = 10^\circ$ (b) $Re = 3 \times 10^6$ and $\alpha = 5^\circ$

Figure 3.3: Representation of the design spaces

It must be noted that the visualisation of the two selected design spaces, given in Figure 3.3 is presented to aid the discussion of results. It was completed after the optimisation algorithm was executed and was not used to train the optimisation algorithm for the specific problem.

Both spaces were examined starting from 20 different initial parameter vectors, examining the performance of the algorithm for initial conditions at regular locations of the design space. For $\alpha = 10°$ 5 % of the cases settled at a local maximum, for $\alpha = 5°$ this result was 90%. The latter can be explained with the presence of two distinctive peaks in this design space, one at slightly lower value than the other. The optima results for the two design spaces are are as follows:

$$f^*_{10°} = \frac{Cl}{Cd} = 110.339 \qquad f^*_{5°} = \frac{Cl}{Cd} = 81.981$$

$$x^*_{10°} = \begin{bmatrix} k \\ \epsilon \end{bmatrix} = \begin{bmatrix} 1.961 \\ 3.037e-2 \end{bmatrix} \qquad x^*_{5°} = \begin{bmatrix} k \\ \epsilon \end{bmatrix} = \begin{bmatrix} 2 \\ 2.6e-2 \end{bmatrix}$$

As expected higher lift to drag ratio is obtained at higher angles of attack, given that the angle of attack is limited well below the critical at which stall occurs. To further validate these results the optimisation path for $\alpha = 10°$ and an initial vector of $k = 1.2, \epsilon = 0.7$ is followed and presented in the following Figure 3.4.

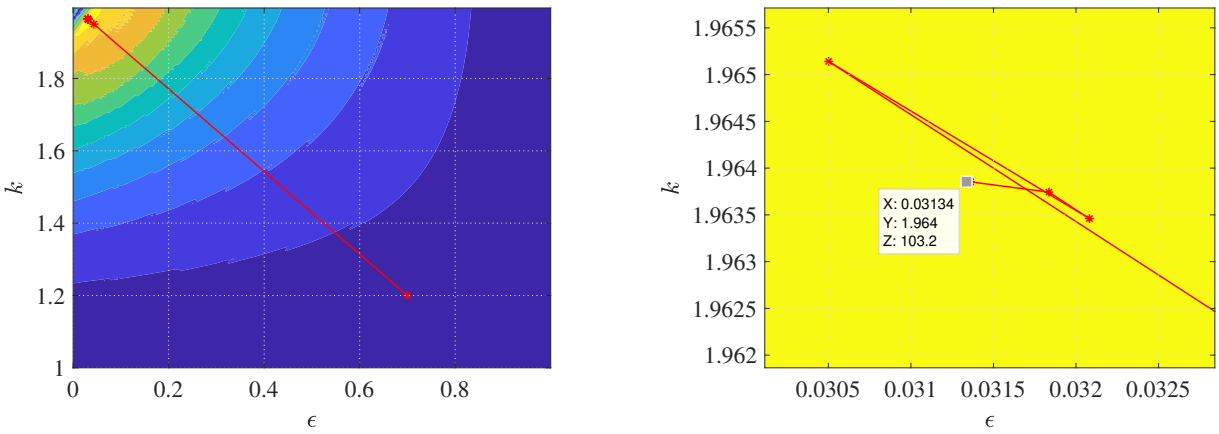
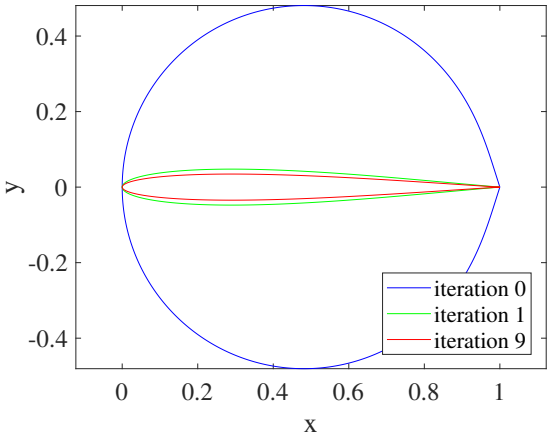
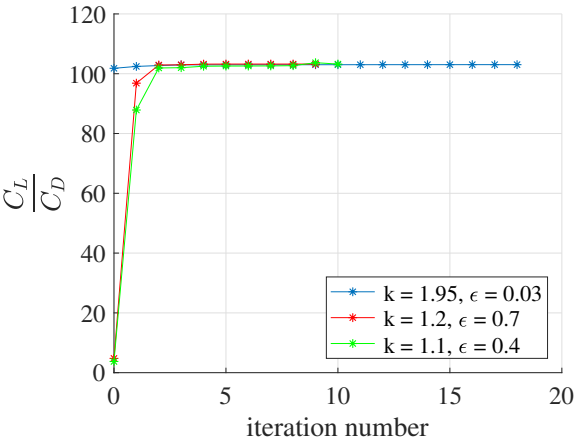


Figure 3.4: Optimisation path for $Re = 3 \times 10^6, \alpha = 10°$ starting from $k = 1.2, \epsilon = 0.7$

We can see that the algorithm successfully reaches the optima, taking 9 iterations to converge. The shape changes occurring during the iterations are represented in Figure 3.5a together with the final geometry which achieves the optima at iteration the 9th iteration. The convergence results for this case are compared to another two cases starting from a point far from the optimum ($k = 1.1$, $\epsilon = 0.4$) and from a point close to it ($k = 1.95$, $\epsilon = 0.03$) and presented in Figure 3.5b. In all cases we can see that a lift to drag ratio close to the optimum is obtained rapidly, however convergence is reached slowly, as expected in limited memory quasi-Newton algorithms [5].



(a) Shape iterations following from an initial vector of $k = 1.2, \epsilon = 0.7$



(b) Convergence of $\frac{C_L}{C_D}$ with three different initial parameter vectors

Figure 3.5: Optimisation results for $Re = 3 \times 10^6, \alpha = 10°$

3.2.1.1 Comparison with the FDM solver implementation

The same two design spaces ($Re = 3 \times 10^6, \alpha = 5°$ and $Re = 3 \times 10^6, \alpha = 10°$) were explored with L-BFGS-B algorithm implementing the FDM solver instead with a pre-selected step size of $\Delta = 10^{-4}$ for the calculation of the gradient. The following plot compares the convergence behaviour of the algorithm in both cases for an initial input parameters $k = 1.2$, $\epsilon = 0.7$.

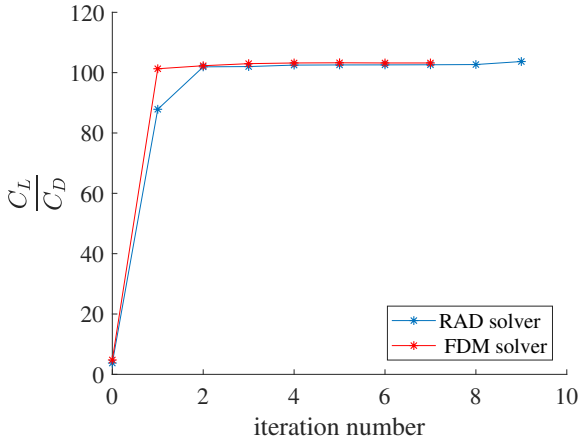


Figure 3.6: Comparison between the convergence behaviour of the FDM and RAD solver implemented in the L-BFGS-B algorithm starting from initial parameter values $k = 1.2$, $\epsilon = 0.4$

Interestingly, although a better performance is expected by the RAD solver, both L-BFGS-B implementations behave very similarly. Further investigation shows that the RAD solver requires 26s to evaluate a single gradient, where FDM solver requires only 11.8s. These results seems surprising at fist sight, however one must consider the overhead in executing

the reverse AD differentiated code. The RAD solver is build on the Store-All approach for memory allocation due to the use of TAPENADE. A review of [11] shows that this approach requires computational time for memory allocation roughly square the run-time of the non-differentiated original solver.

In the two-dimensional case the computational time of evaluating a single gradient for the RAD solver involves a single run of the original code, followed by a run of the differentiated code with additional time for memory allocation. On the other hand, the FDM solver requires only three runs of the original code. For this reason no statement can be made for the run-time savings from the RAD solver in general as the two-dimensional case does not allow for the full potential of the RAD solver to be investigated.

3.2.1.2 Comparison with MATLAB results

To validate the performance of the L-BFGS-B algorithm, an equivalent FDM solver was constructed in MATLAB, based on an implementation by [17], and coupled with the "fmincon" MATLAB routine. For this investigation "fmincon" was set to an implementation of Sequential programming optimisation (SQP) algorithm which allows for the use of quasi-Newton updating method for constrained optimisation [33]. A general overview of SQP methods can be found in [6] and [34]. The following results were obtained with f^* denoting the optimal values of the lift-to-drag ratio found in each test.

α	k	ϵ	**MATLAB: fmincon using SQP**			**Fortran 90: L-BFGS-B with RAD solver**		
			f^*	iterations	functional evaluations	f^*	iterations	functional evaluations
5°	1.95	0.03	85.43	7	22	77.87	5	53
5°	1.2	0.7	85.43	8	37	83.11	5	71
10°	1.95	0.03	103.23	19	121	103.23	13	72
10°	1.2	0.7	103.23	19	123	103.23	9	111

Table 3.2: Comparison between the results obtained using a MATLAB FDM based solver with an SQP algorithm and the developed program, incorporating a RAD solver within an L-BFGS-B algorithm

We can see that the results of both algorithms agree strongly, however there is slight difference in the obtained optima and the L-BFGS-B again demonstrates very slow convergence. Examining the design space more closely shows the presence of multiple optima in the vicinity of the peaks shown in Figure 3.3. Furthermore, for $\alpha = 5°, k = 1.95, \epsilon = 0.03$ the value of the ratio suggests the L-BFGS-B algorithm has terminated at a very low local optima. The superior behaviour of the SQP implementation suggests further avenues of research in exploration of the most appropriate optimisation algorithm in terms of the design space provided. No work, however was carried on in this direction as the choice of algorithm would strongly depend on the selected flow model and the design space. The project re-

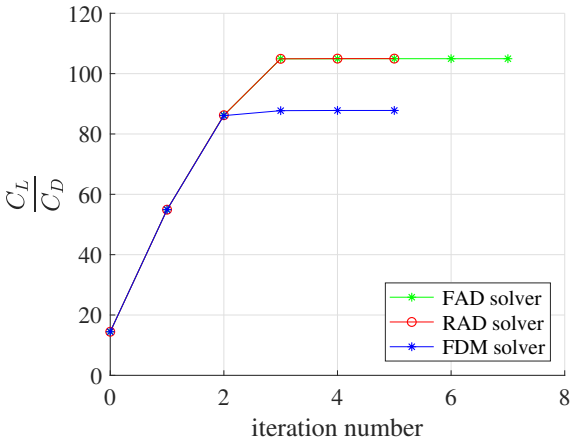
mained on course to explore the applicability of the RAD solver within the optimisation algorithms and avenues of further investigation are suggested in Chapter 4.

3.2.2 Optimisation in six dimensions

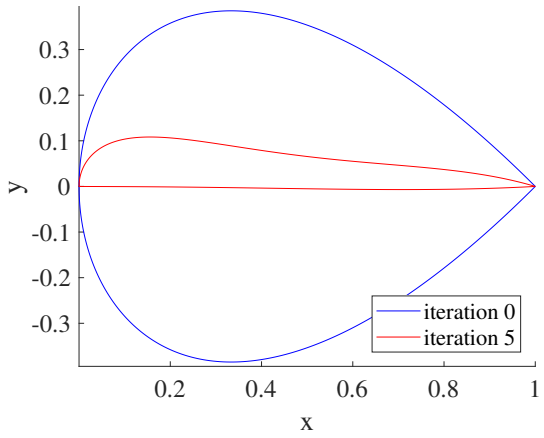
To further observe the performance of the RAD solver in the L-BFGS-B framework and investigate the issues of overhead observed in the two-dimensional case, a second test study was conducted. A six dimensional design space was defined by the CST parametrisation with a input parameter vector $\mathbf{x} = \mathbf{w} \in R^{10}$ and constraints as presented in Section 2.4.3.2. The flow conditions were again chosen to be equivalent to the 2D case with $Re = 3 \times 10^6$ and $\alpha = 10°$. For an example initial vector of $\mathbf{w} = [-1,\ -1,\ -1,\ 1,\ 1,\ 1]^T$ the following results were obtained:

	Iterations	Functional evaluations	Total run time	Optimal $\frac{C_L}{C_D}$
FDM solver	5	55	5min 6s	87.79
FAD solver	7	61	21min 36 sec	104.948
RAD solver	5	66	18min 40 s	104.979

Table 3.3: Comparison of the L-BFGS-B algorithm performance with different solvers



(a) Convergence of $\frac{C_L}{C_D}$ with three different solvers



(b) Shape iteration during optimisation with the RAD solver

Figure 3.7: Optimisation results for $Re = 3 \times 10^6, \alpha = 10°$ and $\mathbf{w} = [-1,\ -1,\ -1,\ 1,\ 1,\ 1]^T$

To obtain the results presented in Figure 3.7 and Table 3.3, a preliminary study of appropriate perturbation step size for the FDM solver was conduced and resulted in the choice of $\Delta = 10^{-4}$. On first glance the result shows the the FDM solver reaches an optima the fastest. Further investigation however reveals that this is due to the termination at a local minima, whereas both the FAD and RAD solver have obtained a higher value for

the optima. This is potentially due to error in the derivative approximation present with the FDM method resulting in less accurate value and direction of the gradient. The FAD and RAD solver in comparison compute the exact values of the partial derivatives and as expected the RAD solver allows for a faster termination in only 5 iterations and a total of 18 min and 40s. The computational savings achieved using the RAD solver in this test case have overcome the overhead issue seen previously under the van der Vooren mapping.

Further investigation, however shows that even with the use of the RAD and FAD solver the L-BFGS-B algorithm also terminated at a local maximum. A run of the L-BFGS-B algorithm using the RAD solver and initial parameter closer to the expected optimum airfoil shape ($\mathbf{w} = [-0.1294,\ -0.0036,\ -0.067,\ 0.206,\ 0.2728,\ 0.229]^T$) obtains the following result:

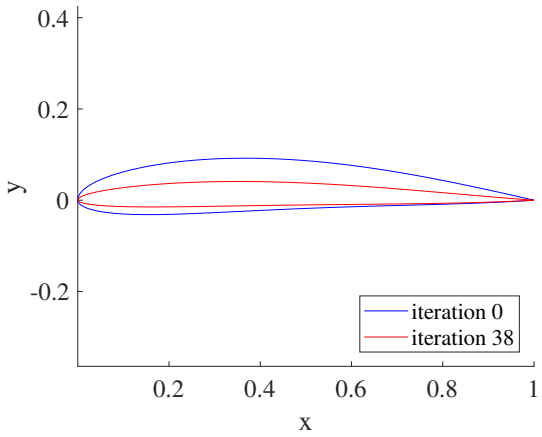
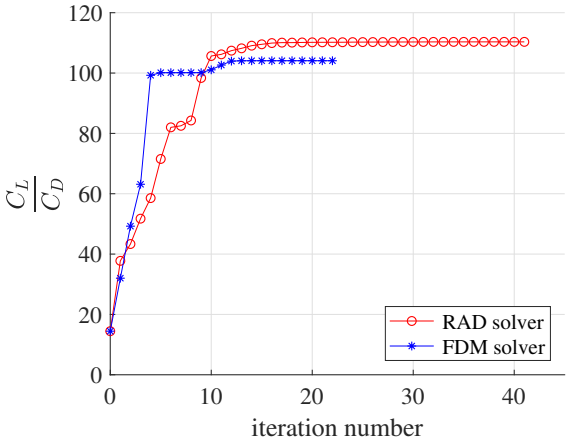


Figure 3.8: Shape iteration using the RAD solver with a new initial parameter vector

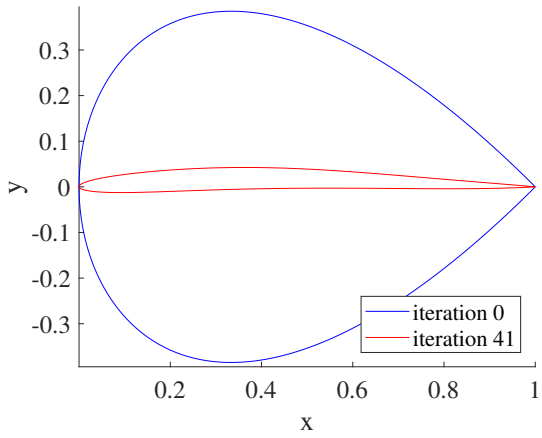
This final shape achieves a lift to drag ratio of 108.588 in 38 iterations for a total run time of 77min and 40s. Again, there are no guarantees that this is the optimal. In fact in the following case with a ten-dimensional input vector we achieve a higher value. All these results further supports the case presented previously, for the need of further investigation of the most appropriate optimisation algorithm and demonstrate a well known problem of finding the global optimum in non-convex functions. A collection of the most recent advances in the area of global optimisation can be found in [35].

3.2.3 Optimisation in ten dimensions

To further support these results a similar study was conducted with a ten-dimensional input vector, again defined by the CST-parametrisation and the constraints given in Section 2.4.3.2. For $Re = 3 \times 10^6$, $\alpha = 10^\circ$ and an input vector of
$\mathbf{w} = [-1,\ -1,\ -1,\ -1,\ -1,\ 1,\ 1,\ 1,\ 1,\ 1]^T$ the obtained results are presented in Figure 3.9.



(a) Convergence of $\frac{C_L}{C_D}$ with two different solvers



(b) Shape iteration during optimisation with the RAD solver

Figure 3.9: Optimisation results for $Re = 3 \times 10^6, \alpha = 10°$

After a preliminary study of the most appropriate perturbation step size, similar to the one presented in Section 3.1.1.3, the FDM solver was initialised with $\Delta = 10^{-4}$. As we can see it has again terminated at a local maximum, lower than the value obtained with the RAD solver. The RAD implementation reaches an optima of 110.339 in 41 iterations,with a total run-time of 70 min 3s. The FDM solver in comparison reaches only 104.087 in 82 min and 42 sec despite terminating after only 22 iterations.

Two main points arise from the comparison of the RAD performance and FDM performance in the 2-dimensional, six-dimensional and 10-dimensional case. First, an overview of run time shows that the run-time of a single gradient calculation remains 26 seconds for the RAD solver in all three cases. The FDM solver however shows a significant increase from 11.8s to 36s from the two- to the ten-dimensional case respectively. Furthermore, it requires additional time to determine an appropriate perturbation step size and reaches only local maximums in cases where RAD and FAD solvers clearly reach higher values for a shorter run-time as demonstrated above.

A second important point to highlight is the higher value obtained in the 10-dimensional optimisation than both the two-dimensional and six-dimensional cases. For the two-dimensional case this is expected as the van der Vooren allows for only symmetrical airfoil shapes. For the 6-dimensional case, however, this shows that even the RAD solver implementation terminates at a local maximum. Furthermore, no guarantee is present that the 10-dimensional case achieves the global maximum as well. In the convergence plot we see that it does not reach the optimal as steadily as the two-dimensional and six-dimensional case, which speaks to the presence of multiple local optima.

These results highlights an important trade off. In both the six- and ten-dimensional cases the single gradient evaluation takes 26 seconds but the overall time of the algorithm has

increased significantly compared to the two-dimensional case. The number of local maxima as well as the time to convergence depend strongly on the design space. Increasing the number of input parameters has led to the finding of higher values for the optimal due richer design space, but has increased the frequency of local optima findings as well the computation time.

Nevertheless, the results however show the superiority of the RAD solver in comparison to the FDM solver for higher number of input parameters. The improved accuracy in the derivative computation with the RAD solver results in more accurate search directions and constant run-time per single gradient calculation regardless of the dimension of the input vector.

Chapter 4

Future work

Several avenues for further research were outlined through this report and are summarised here for completeness.

4.1 Model improvement

The main aim of this project was to conduct an initial investigation of the applicability of reverse automatic differentiation to the framework of gradient-based shape optimisation for airfoils. As such, the flow solver is based on a simplified flow model, developed with pedagogical aims for third year students in the Cambridge Engineering Department as part of a Wing Analysis Project. Although modified for the purposes of this project, as detailed in Chapter 2, the model has several limitations.

The current flow solver is unable to accurately model the fluid flow at high angles of attack. Avenues for further work include different techniques of coupling between the inviscid and viscous solutions, which are summarised by [32] for 2D models. Additionally, due to the implementation of 2D panel method a compressibility correction has to be applied to the incompressible results in order to provide an accurate model at high subsonic Mach numbers. Several different corrections have been developed for 2D flowfield models in the speed region before the onset of transonic effects, such as wave drag resulting from local supersonic flow [36]. Some of the possibilities include the Prandtl-Glauert correction rule [37], Laitone rules [38] or the complex Kármán–Tsien relationship [39, 40].

Although, the 2D model provides a good approximation of the flow field, it fails to accommodate some key effects, which are observed once a three-dimensional flow is considered. The model would need to be extended to include the effects due to the finite span of the wing. This includes the presence of spanwise pressure gradients, which cause cross flows resulting in the shedding of a sheet of streamswise vorticity at the trailing edge and giving rise to a phenomenon known as induced drag [41].

4.2 Gradient calculation

As mentioned in Section 1.5 avenues for further work include a hybrid approach for the computation of the gradient using reverse AD selectively within the framework of adjoint methods. This approach results in reduced memory storage requirements in comparison to the direct use of reverse AD to compute the gradient and improved time complexity compared to the finite difference method of estimating the partial derivatives building the gradient.

4.3 Optimisation algorithm

As mentioned several times previously the choice of optimising algorithm depends strongly on the design space. As seen in Section 3.2, different algorithms can be exploited to obtain better overall complexity of the framework. This work was not conducted within the current project due to time limitation. It must be noted however, that implementation of any of the model improvements outlined above would result in a different design space, for which the appropriate optimisation algorithm might be different.

Chapter 5

Conclusion

In this project we developed three different solvers for the calculation of an airfoil's lift to drag ratio and its gradient with respect to the input parameters, defining its geometry. Their performance was investigated through studies of the sensitivity of the stream function and the lift-to-drag ratio to perturbations in the input geometry parameters. The results showed that in the calculation of the partial derivatives, the FDM solver introduces an error which is strongly dependent on the perturbation step size, whereas the FAD solver eliminates both truncation and subtractive cancellation errors to provide an exact value of the derivative.

Furthermore, implementation of each of the solvers within the L-BFGS-B algorithm resulted in successful localisation of local maxima of the lift-to-drag ratio within three different design spaces. In the case of two-dimensional van de Vooren parametrisation, the RAD solver demonstrated longer run time per single gradient calculation than the FDM solver due to the overhead introduced by the Store-All memory allocation tactic. The superiority of the RAD solver, however, was demonstrated using a larger number of input parameters under the CST parametrisation. For the cases of both six- and ten-dimensional input vectors the RAD solver showed constant run time per gradient evaluation regardless of the number of input parameters, whereas the time required by the FDM solver increased by nearly a factor of three from the two-dimensional case. In addition the implementation of the FDM solver introduced truncation errors in the partial derivatives building the gradient, which resulted in inaccurate search directions and termination of the L-BFGS-B algorithm at local maxima much lower than the values obtained with the implementations of the FAD and RAD solvers.

Regardless of the solver used, the LBFGS- B algorithm showed show convergence and the complexity of the design spaces highlighted the need for further research to select the most appropriate optimisation algorithm for the developed 2D flow model.

Bibliography

[1] F. H. Harlow. Fluid dynamics in group t-3 los alamos national laboratory: (LA-UR-03-3852). *Journal of Computational Physics*, 195(2):414 – 433, 2004.

[2] J.L. Hess and A.M.O. Smith. Calculation of potential flow about arbitrary bodies. *Progress in Aerospace Sciences*, 8:1 – 138, 1967.

[3] M. J. Lighthill. A new method of two-dimensional aerodynamics design. In *Aeronautical Research Council*, 1945.

[4] Ya-Xiang Yuan. A review of trust region algorithms for optimization, 1999.

[5] J. Nocedal and S. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer New York, 1999.

[6] P.E. Gill, W. Murray, and M.H. Wright. Practical optimization. 1981.

[7] W. Squire and G. Trapp. Using complex variables to estimate derivatives of real functions. *SIAM Review*, 40:110–112, 1998.

[8] J. Lyness and C. Moler. Numerical differentiation of analytic functions. *SIAM Journal on Numerical Analysis*, 4(2):202–210, 1967.

[9] J. R. A. Martins, P. Sturdza, and J. J. Alonso. The complex-step derivative approximation. *ACM Trans. Math. Softw.*, 29(3):245–262, September 2003.

[10] M. Jerosolimski and L. Levacher. New method for fast calculation of jacobian matrices: automatic differentiation for power system simulation. *IEEE Transactions on Power Systems*, 9(2):700–706, 1994.

[11] L. Hascoët. Automatic differentiation by program transformation. 2007.

[12] Atilim Gunes Baydin, Barak A. Pearlmutter, and Alexey Andreyevich Radul. Automatic differentiation in machine learning: a survey. *CoRR*, 2015.

[13] M.B. Giles, D. Ghate, and M.C. Duta. Using automatic differentiation for adjoint CFD code development. In *Recent Trends in Aerospace Design and Optimization*, pages 426–434. Tata McGrawHill New Delhi, 2006.

[14] Y. Gao, Y. Wu, and J. Xia. Automatic differentiation based discrete adjoint method for aerodynamic design optimization on unstructured meshes. *Chinese Journal of Aeronautics*, 30(2):611 – 627, 2017.

[15] T. Shanbhag. Adjoint-based shape optimisation of aerofoils. *IIB project report*, 2018. Cambridge University Engineering Department. Unpublished.

[16] L. Hascoët, M. Vázquez, and A. Dervieux. Automatic differentiation for optimum design, applied to sonic boom reduction. In *Proceedings of the International Conference on Computational Science and its Applications, Montreal, Canada*, pages 85–94. LNCS 2668, Springer, 2003.

[17] R. Garcia-Mayoral. SA1: Aircraft wing analysis. 2018. Cambridge University Engineering Department. Unpublished.

[18] Y. Nakayama. *Introduction to Fluid Mechanics*. Elsevier Science, 2018.

[19] L. Hascoët and V. Pascual. The Tapenade Automatic Differentiation tool: Principles, Model, and Specification. *ACM Transactions On Mathematical Software*, 39(3), 2013.

[20] L. Hascoët and V. Pascual. Tapenade 2.1 user's guide. Technical report, INRIA, 2004.

[21] LAPACK Linear Algebra Package. `http://www.netlib.org/lapack/`. Accessed: 2019-05-26.

[22] J. Katz and A. Plotkin. *Low-Speed Aerodynamics*. Cambridge Aerospace Series. Cambridge University Press, 2001.

[23] A. I. van de Vooren and L.S. de Jong. *Calculation of Incompressible Flow about Aerofoils Using Source, Vortex Or Doublet Distribution*. Univ. Mathematisch Inst., 1970.

[24] B. M. Kulfan. A universal parametric geometry representation method "CST". *AIAA Paper 2007-0062*, pages 1–45.

[25] M. A. Ceze, M. A. Hayashi, and E. Volpe. A study of the CST parameterization characteristics. 2009.

[26] R. EPPLER and D. M Somers. A computer program for the design and analysis of low-speed airfoils. *NASA Technical Memorandum*, 1980.

[27] A.D. YOUNG. Boundary layers. page 656–657, 1989.

[28] M. Giles. An extended collection of matrix derivative results for forward and reverse mode algorithmic dieren tiation. 2008.

[29] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal. Algorithm 778: L-bfgs-b: Fortran subroutines for large-scale bound-constrained optimization. *ACM Transactions on Mathematical Software*, 23(4):550–560, 1 1997.

[30] R.M. Pinkerton E.N. Jacobs, K.E. Ward. The effect of compressibility on the lift of an aerofoil. *NACA Report*, (460), 1933.

[31] I.H. Abbott and A.E. Von Doenhoff. *Theory of Wing Sections, Including a Summary of Airfoil Data*. Dover Books on Aeronautical Engineering Series. Dover Publications, 1959.

[32] H. Bijleveld and A. Veldman. Prediction of unsteady flow over airfoils using a quasi-simultaneous interaction method. 05 2019.

[33] MathWorks: Constrained Nonlinear Optimization Algorithms. `http://web.archive.org/web/20080207010024/http://www.808multimedia.com/winnt/kernel.htm`. Accessed: 2019-05-28.

[34] O. Exler, T. Lehmann, and Klaus Schittkowski. A comparative study of sqp-type algorithms for nonlinear and nonconvex mixed-integer optimization. *Mathematical Programming Computation*, 4(4):383–412, Dec 2012.

[35] D. Gao, N. Ruan, and W. Xing. *Advances in Global Optimization*. Springer Proceedings in Mathematics & Statistics. Springer International Publishing, 2015.

[36] T. Melin, A. T. Isikveren, and M. I. Friswell. Induced-drag compressibility correction for three-dimensional vortex-lattice methods. *Journal of Aircraft*, 47(4):1458–1460, 2010.

[37] H. Glauert and G. I. Taylor. The effect of compressibility on the lift of an aerofoil. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, 118(779):113–119, 1928.

[38] E. V. Laitone. New compressibility correction for two-dimensional flow. *Journal of the Aeronautical Sciences*, 18:350, 05 1951.

[39] T. von Kármán. Compressibility effects in aerodynamics. *Journal of the Aeronautical Sciences*, 8:337–356, 07 1941.

[40] H.S. Tsien. Two-dimensional subsonic flow of compressible fluids. *Journal of the Aeronautical Sciences*, 6:399–407, 1939.

[41] 3A1 Applications I: Aerofoils and Wings. *Lecture notes*, 2016. Cambridge University Engineering Department. Unpublished.

Appendix A

Influence coefficients

Figure A.1 shows an single vortex-sheet panel with linear varying strength from γ_1 and γ_2 of length Δ.

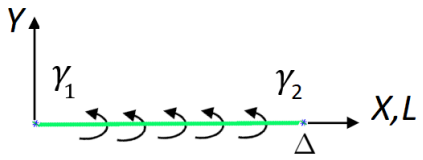


Figure A.1: A single panel notation

The stream function ψ resulting from the panel at a general location (X,Y) can be expressed as:

$$\psi(X,Y) = \gamma_1 f_1 + \gamma_2 f_2 \tag{A.1}$$

wheref_1 and f_2 are known as influence coefficients and can be computed in the following manner as presented in [17]:

$$f_1 = \Big[\Big(L - \frac{X}{\Delta}\Big)I_0 - \frac{I_1}{\Delta}\Big] \qquad f_2 = \Big[\frac{X}{\Delta}I_0 + \frac{I_1}{\Delta}\Big] \tag{A.2}$$

where

$$I_0 = -\frac{1}{4\pi}\int_0^{\Delta} log\Big[(L-X)^2 + Y^2\Big]dL = \tag{A.3}$$

$$= -\frac{1}{4\pi}\Big\{X log[X^2+Y^2] - (X-\Delta)log[(X-\Delta)^2+Y^2] - ...$$

$$...2\Delta + 2Y\Big[tan^{-1}\Big(\frac{X}{Y}\Big) - tan^{-1}\Big(\frac{X-\Delta}{Y}\Big)\Big]\Big\}$$

$$I_1 = -\frac{1}{4\pi}\int_0^{\Delta}(L-X)log\Big[(L-X)^2+Y^2\Big]dl = \tag{A.4}$$

$$= \frac{1}{8\pi}\Big\{[X^2+Y^2]log[X^2+Y^2] - [(X-\Delta)^2+Y^2]log[(X-\Delta)^2+Y^2] - 2X\Delta + \Delta^2\Big\}$$

Appendix B

Laminar regime

For a laminar boundary layer the solver implements Thwaites' solution to the momentum integral equation as presented in [17]:

$$\Big[\frac{\theta(x)}{L}\Big]^2 = \frac{0.45}{Re_L}\Big[\frac{u_e(x)}{U}\Big]^{-6}\int_0^{x/L}\Big[\frac{u_e(x')}{U}\Big]^5 d\Big(\frac{x'}{L}\Big) \tag{B.1}$$

where θ denotes momentum thickness. L and U are the reference length and velocity respectively on which the Reynolds number Re_L is defined. u_e is the velocity at the edge of the boundary layer, which is obtained from the estimated pressure distribution by the potential flow solver at each panel edge i as follows:

$$\frac{u_{e,i}}{U} = \sqrt{1 - c_{pi}} \tag{B.2}$$

Numerical integration as presented in [17] is achieved through summation of the contribution to the integral from each panel with edge coordinates $x_i = x'/L$ and $x_{i+1} = x'/L$. Assuming linear variation in u_e each contribution to the integral in Equation B.1 can be estimated as follows

$$\int_{x_i}^{x_{i+1}}\Big[\frac{u_e(x')}{U}\Big]^5 d\Big(\frac{x'}{L}\Big) = \Big[\overline{u}^5 + \frac{5}{6}\overline{u}^3(\Delta u)^2 + \frac{1}{16}\overline{u}(\Delta u)^4\Big]\Delta x \tag{B.3}$$

$$\overline{u} = \frac{u_i + u_{i+1}}{2} \tag{B.4}$$

$$\Delta u = u_{i+1} - u_i \qquad\qquad \Delta x = x_{i+1} - x_i \tag{B.5}$$

The summation is carried out until a transition to turbulence or separation is detected. Both checks depend on a the gradient parameter m defined as

$$m = -Re_L\Big(\frac{\theta}{L}\Big)^2\frac{d(u_e/U)}{d(x/L)} \tag{B.6}$$

Flow separation is predicted separation once m increases beyond 0.09, following a criterion based on Thwaites' method [17]. Furthermore, the gradient parameter can be related to the shape factor H through a tabulated relationship by Thwaites [17]. The shape factor can then be used to estimate the energy shape factor H_E using a relationship presented by Eppler and Somers [26]. This leads to the following empirical criterion for transition to turbulence again proposed by Eppler and Somers:

$$log(Re_\theta) \geq 18.4H_E - 21.74 \tag{B.7}$$

where Re_θ denotes the local Reynolds number based on momentum thickness defined as

$$Re_\theta = Re_L\frac{u_e}{U}\frac{\theta}{L} \tag{B.8}$$

Finally, if laminar separation has occurred Epplers and Sommers predict the flow will reattach once H_E exceeds 1.58.

Appendix C

Turbulent regime

Once transition is detected the code implements a numerical solution of the momentum and kinetic integral equations as presented in [17]. The momentum integral equation can be stated as

$$\frac{d\theta}{dx} + \frac{H+2}{u_e}\frac{du_e}{dx}\theta = \frac{c_f}{2} \tag{C.1}$$

where θ is the momentum thickness, H is the shape factor and c_f is the skin friction coefficient. u_e represents the velocity at the edge of the boundary layer and its estimation was presented in Appendix B. The kinetic boundary equation can be stated as

$$\frac{d\delta_E}{dx} + \frac{3}{u_e}\frac{du_e}{dx}\delta_E = c_{diss} \tag{C.2}$$

where δ_E represent the energy thickness and c_{diss} denotes the energy dissipation coefficient. These can be rewritten in the following form, as used by the integral solvers:

$$\begin{bmatrix} \dfrac{d\theta}{dx} \\ \dfrac{d\delta_E}{dx} \end{bmatrix} = \begin{bmatrix} \dfrac{c_f}{2} - \dfrac{H+2}{u_e}\dfrac{du_e}{dx}\theta \\ c_{diss} - \dfrac{3}{u_e}\dfrac{du_e}{dx}\delta_E \end{bmatrix} \tag{C.3}$$

The values of c_f, c_{diss} and H can be calculated through the following empirical closure relations proposed by Eppler and Somers [26]:

$$H = \frac{11H_E + 15}{48H_E - 59} \qquad H_E \geq 1.46 \tag{C.4}$$

$$H = 2.803 \qquad H_E < 1.46 \tag{C.5}$$

$$c_f = 0.091416[(H-1)Re_\theta]^{-0.232}e^{-1.260H} \tag{C.6}$$

$$c_{diss} = 0.010018[(H-1)Re_\theta]^{-1/6} \tag{C.7}$$

H_E denotes the energy shape factor and Re_θ is the Reynolds number based on momentum thickness as defined as previously in Appendix B. Eppler and Somers predict turbulent separation once H_E falls below 1.46 in which case the relationships C.4,C.5,C.6 and C.7 are no longer valid.

Under these flow conditions $c_f = 0$ and the momentum integral C.1 can be shown to result in

$$\left(\frac{\theta_{i+1}}{\theta_i}\right) = \left(\frac{u_{e,i+1}}{u_{e,i}}\right)^{(H+2)} \tag{C.8}$$

where u_{ei} represents the boundary edge velocity at the the ith panel edge and analogously θ_i is the momentum thickness at that location [17].

Appendix D

Risk assessment retrospective

There were no unexpected risks or hazards encountered during this project. The work was entirely based on a computer and in retrospect no additions to the completed risk assessment were necessary.